

EVOLVER: Online Learning and Prediction of Disturbances for Robot Control

Jindou Jia , Wenyu Zhang , Kexin Guo , Jianliang Wang , Senior Member, IEEE, Xiang Yu , Senior Member, IEEE, Yang Shi , Fellow, IEEE, and Lei Guo , Fellow, IEEE

Abstract—In nature, when encountering unexpected uncertainty, animals tend to react quickly to ensure safety as the top priority, and gradually adapt to it based on recent valuable experience. We present a framework, namely EVOLutionary model-based uncertainty obserVER (EVOLVER), to mimic the bio-behavior for robotics to achieve rapid transient reaction ability and high-precision steady-state performance simultaneously. In particular, the Koopman operator is leveraged to explore the latent structure of internal and external disturbances, which is subsequently utilized in an evolutionary model-based disturbance observer to estimate the eventual disturbance. The resulting observer can guarantee a provable convergence in optimal conditions. Several practical considerations, including construction of a training dataset, data noise handling, and lifting functions selection, are elaborated in pursuit of the theoretical optimality in real applications. The lightweight feature of our framework enables online computation, even on a microprocessor (STM32F7 with 100 Hz control frequency). The framework is thoroughly evaluated by one simulation and three experiments. The experimental scenarios include: 1) Trajectory prediction of an irregular free-flying object subject to aerodynamic drag, 2) indoor and outdoor agile flights of a quadrotor subject to

wind gust, and 3) high-precision end-effector control of a manipulator subject to base moving disturbance. Comparison results show that the performance of our proposed EVOLVER is superior to several state-of-the-art model-based and learning-based schemes.

Index Terms—Disturbance observer, disturbance prediction, Koopman operator, online learning for control.

I. INTRODUCTION

ANALYTICALLY established dynamical models seldom conform exactly to the control demand in real-world robotics applications. This is especially true for a nonlinear robotic system operating in an unstructured environment (e.g., a quadrotor maneuvering in an unknown windy field or a manipulator operating on a floating base). Without any doubt, how to eliminate system disturbances/uncertainties is one of the central tasks in the robot control community.

In the traditional control community, a plethora of *model-based* schemes have been designed to tackle different uncertainties in the past decades, such as adaptive control [1], H_∞ control [2], sliding mode control (SMC) [3], neural network-based control [4], and composite hierarchical antidisturbance control [5]. A sudden unexpected uncertainty can be quickly (not completely) suppressed at the transient stage by real-time feedback or feedforward methods, ensuring the initial safety of the robot. However, focusing on uncertainty with less a priori information, it is difficult to eliminate the adverse effect completely in the steady state, inducing a bounded convergence set [6].

In sharp contrast, the *data-driven-based* techniques are leveraged to fit the uncertainty offline or online [7] in the learning community. The final approximation accuracy can achieve decent performance when the learned data is enough and the designed policy is efficient. Despite of being appealing, since the learning procedure takes time to construct the initial training dataset and train the model [7], the ability to promptly respond to unexpected uncertainty is limited, which may result in an unsafe operation initially [8]. Whereas initial safety is crucial for many robotics, e.g., a quadrotor maneuvering under an abrupt wind disturbance has to make sure that it will not be blown over initially. Moreover, the stability of most learning methods is difficult to guarantee from a theoretical perspective [9]. Another ever-rising learning strategy is to explore the latent uncertainty structure offline and learn the eventual uncertainty adaptively online.

Manuscript received 1 June 2023; accepted 9 October 2023. Date of publication 20 October 2023; date of current version 15 December 2023. This paper was recommended for publication by Associate Editor L. Righetti and Editor P. Robuffo Giordano upon evaluation of the reviewers' comments. This work was supported in part by the Defense Industrial Technology Development Program under Grant JCKY2020601C016, in part by the National Natural Science Foundation of China under Grant 61833013, Grant 61973012, Grant 62388101, Grant 62173024, and Grant 62273023, in part by the National Key Research and Development Program of China under Grant 2022YFB4701301, in part by the Natural Science Foundation of Zhejiang Province under Grant LD21F030001, in part by the Key Research and Development Program of Zhejiang under Grant 2021C03158, in part by the Major Science and Technology Innovation Program of Hangzhou under Grant 2022AIZD0137, in part by the Beijing Nova Program under Grant 20230484266, and in part by the Outstanding Research Project of Shen Yuan Honors College, BUAA under Grant 230122104. (Corresponding author: Xiang Yu.)

Jindou Jia is with the School of Automation Science and Electrical Engineering, Beihang University, Beijing 100191, China, and also with the Shenyuan Honors College, Beihang University, Beijing 100191, China (e-mail: jidjia@buaa.edu.cn).

Wenyu Zhang, Xiang Yu, and Lei Guo are with the School of Automation Science and Electrical Engineering, Beihang University, Beijing 100191, China (e-mail: wyzhang@buaa.edu.cn; xiangyu1110@gmail.com; lguo@buaa.edu.cn).

Kexin Guo is with the School of Aeronautic Science and Engineering, Beihang University, Beijing 100191, China (e-mail: kxguo@buaa.edu.cn).

Jianliang Wang is with the Collaborative Flying Robotics, Hangzhou Innovation Institute, Beihang University, Hangzhou 310051, China (e-mail: wjl-180@hotmail.com).

Yang Shi is with the Department of Mechanical Engineering, University of Victoria, Victoria, BC V8W 3P6, Canada (e-mail: yshi@uvic.ca).

This article has supplementary material provided by the authors and color versions of one or more figures available at <https://doi.org/10.1109/TRO.2023.3326318>.

Digital Object Identifier 10.1109/TRO.2023.3326318

An interesting question arises: Is it possible to possess rapid transient reaction ability and high-precision steady-state tracking performance simultaneously in one interpretable framework? This article provides an affirmative answer to this challenge. In nature, the behavior of integrating reactive control and active learning/prediction for creatures has been investigated. In [10], researchers study the optomotor response of the larval zebrafish under multiple perturbations. At the transient stage, the feedback reactive control with a proven integration part can deal with abrupt perturbations. If the suffered perturbation is long-lasting and predictable, the larval zebrafish can learn the new environmental condition in the cerebellum and induce no more perception delay.

Inspired by the bio-behavior, we present an **EVOLutionary model-based uncertainty obserVER** (EVOLVER) by merging reactive control and active learning to achieve the abovementioned objective. At the transient stage, the initial uncertainty observer can guarantee rapid reaction performance without a priori uncertainty information. By virtue of the learned model of uncertainty, the uncertainty observer can achieve high-precision estimation in the steady state. Evolutionary contains two meanings. One is that the uncertainty observer in the steady state is evolutionary relative to the one at the transient stage with the help of the learned uncertainty model. The other one is that the learned uncertainty model can continue to evolve with the arrival of additional evaluated valuable data.

A. Contributions

The main contributions are summarized as follows.

In Section IV, a framework of online learning and prediction of disturbances for robot control, that appropriately integrates *model-based* estimation with *data-driven-based* learning, is presented to handle uncertainties. The EVOLVER is able to achieve rapid transient reaction ability and high-precision steady-state tracking performance simultaneously. The zero-error convergence analysis of the EVOLVER is provided in optimal conditions: the uncertainty is predictable, infinite independent data samples, and orthonormal lifting functions. For uncertainties influenced by unknown factors (i.e., unpredictable), it is intractable to learn accurately, whereas the adverse effect can be alleviated by further integrating historical states of uncertainty into the lifting functions, as presented in Section IV-E.

Practical considerations, including training dataset, data noise, and lifting functions selection that affect estimation performance, are analyzed in pursuit of theoretical optimality. 1) A data selection method considering the *timeliness* and *correlation* of the data is designed in Section V-A, to make the dataset cover more learning-promoting samples. 2) A robust differentiator (RD) with a predefined convergence time is exploited to construct a noiseless and accurate training dataset, as presented in Section V-B. 3) Manual and automatic selection strategies for the lifting functions are utilized in response to different scenarios, as discussed in Section V-C. Our framework enables online computation, even on a STM32F7 with 100 Hz control frequency.

Extensive evaluations in various scenarios are conducted to demonstrate the applicability of the EVOLVER in Section VI. In simulation, the chaotic *Lorenz* uncertainty for a second-order *Newton* system is studied. In experiments, as shown in Fig. 1, trajectory prediction of an irregular free-flying object under aerodynamic drag, indoor and outdoor agile flights of a quadrotor under wind disturbance, and end-effector control of a manipulator under base disturbance are implemented.

One interesting finding is that some common characteristics exist among the proposed EVOLVER and the larval zebrafish [10]. Both of them can not only completely overcome the uncertainty at the transient stage when the suffered disturbance is constant, but also eliminate action delays in the steady state after the learned uncertainty model is employed.

B. Organization and Notation

Organization: The rest of this article is organized as follows. Section II surveys related work. Section III formulates the uncertainty learning problem. Section IV presents the main theoretical results. Section V details several specific practical considerations. The applicability of the EVOLVER is demonstrated in one simulation and three experiments in Section VI. Finally, Section VII concludes this article.

Notation: Throughout the article, $\mathbb{R}$ denotes the real set; the column vector is denoted with a bold lowercase letter (e.g., $\mathbf{a}$), and the matrix is denoted with bold capital (e.g., $\mathbf{A}$); $\lambda_M(\cdot)$ represents the maximum eigenvalue of a symmetric matrix; $\circ$ denotes the composition operation; $\mathbf{A}^\dagger$ denotes the Moore–Penrose pseudoinverse of $\mathbf{A}$; $\|\cdot\|$ denotes the 2-norm of a vector; $\hat{\mathbf{a}}$ denotes the estimation of $\mathbf{a}$ and $\tilde{\mathbf{a}}$ denotes the estimation error $\tilde{\mathbf{a}} = \mathbf{a} - \hat{\mathbf{a}}$.

II. RELATED WORK

Broadly, the literature on handling uncertainty can be classified into two categories: *attenuation* by robust control schemes and *compensation* by a hierarchical controller equipped with specific uncertainty estimation techniques [11]. A concise review is presented below.

A. Uncertainty Attenuation Scheme

Attenuation methods usually consider the worst case that the dynamic system may suffer from, resulting in performance conservativeness. Robust SMC [3] utilizes a sign function term related to the upper bound of disturbance, to attenuate the disturbance. Prescribed performance control (PPC) [12] guarantees a predefined convergence rate and arbitrarily small convergence residual set by a series of error transformations. In essence, PPC amplifies the small error at the beginning and uses larger control input in consequence. The ideal antidisturbance performances of SMC and PPC depend on the extremely fast response speed and unlimited amplitude of the actuator. This assumption limits their applications in real robotics with physical constraints. Robust H_∞ control [2] aims to suppress the H_∞ norm of the transfer

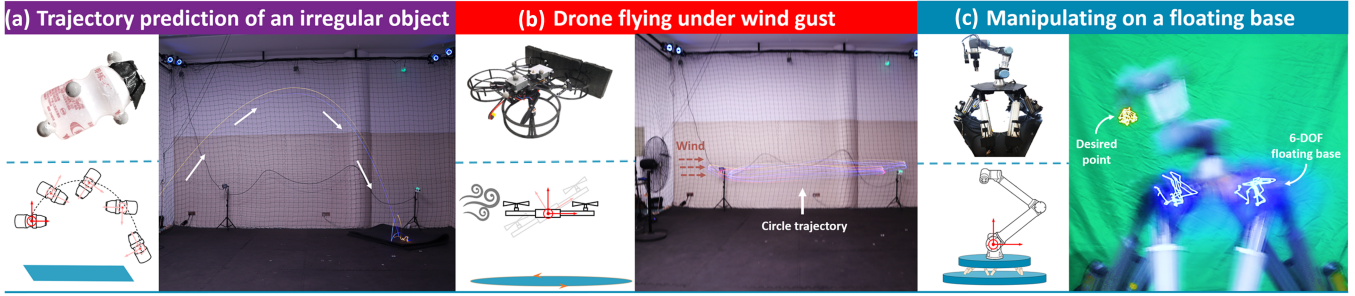


Fig. 1. We demonstrate our approach through (a) trajectory prediction of an irregular free-flying object under aerodynamic drag, (b) agile flight of a quadrotor under wind disturbance, and (c) end-effector control of a manipulator under unwanted base moving disturbance. Long-exposure photos of the above three tasks are taken by placing several LED lamps on the experimental subjects. Experimental video is available in the supplementary materials <https://sites.google.com/view/buaa-evolver>.

function from disturbance/uncertainty to output. The *attenuation* structure is a single-DOF control, where contradictions exist among different performance requirements (e.g., tracking *versus* disturbance rejection, nominal performance *versus* robustness) [13]. In addition, the *attenuation* scheme works after the disturbance has induced output error, namely, a postreaction scheme, which is inferior to the prereaction scheme (i.e., feedforward *compensation*).

B. Uncertainty Compensation Scheme

The basic idea behind *compensation* methods is to employ an observer to estimate the uncertainty by way of its influence on the system performance, and subsequently compensate its influence by using feedforward control. This control scheme is able to act before uncertainty induced error is present as long as the prediction is accurate. *Compensation* method is a two-DOF control [5], [14], providing a promising solution for addressing the drawbacks existing in most robust control techniques [13]. With proper integration with a baseline controller, the tracking and uncertainty rejection performances can be achieved simultaneously. There are mainly two design philosophies for the observer: *model-based* construction and *data-driven-based* learning. *Model-based* observers utilize the system model and a priori information of uncertainty to construct a convergent observer. In contrast, *data-driven-based* ones intend to learn the uncertainty in terms of datasets extracted from the robotic system.

1) *Model-Based Methods*: In the last four decades, extensive *model-based* uncertainty observer schemes have been developed. With respect to linear systems, frequency domain disturbance observer [15], extended state observer (ESO) [16], unknown input observer (UIO) [17], generalized proportional-integral observer (GPIO) [18], and disturbance prediction observer [14] are presented. Extended high-gain state observer [19] and time domain nonlinear disturbance observer (TDNDO) [20] are designed for nonlinear systems. Although different methods are developed independently, the ideas behind the results are similar. For example, under the proper selection of observer parameters, ESO, UIO, and GPIO are equivalent [13]. Most disturbance observers can achieve zero estimation error in the event of constant disturbances.

For more complicated disturbances, *model-based* disturbance observer usually requires a priori disturbance features. For example, UIO [17] and TDNDO [21] can accurately estimate the disturbance produced by exogenous linear systems, e.g., harmonic disturbance with known frequencies. For time-varying disturbances that can be represented by a high-order polynomial of time, GPIO [18] and higher order TDNDO [22] can achieve zero asymptotic convergence of the disturbance estimation. A feedforward control method is proposed in [14] with disturbance prediction, which assumes the disturbance can be represented by a high-order polynomial of time series or the historical states. For multidisturbances, simultaneous attenuation and compensation mechanism is proposed in [5]. In [4] and [23], neural networks (NNs) are exploited to represent uncertainty, where the weights are adjusted by an adaptive feedback scheme online. However, the adaptive NN-based scheme cannot handle external disturbances independent of the system states. Note that we classify such adaptive NN-based methods as *model-based* observer because no data-mining of historical data is involved and the selection of basis functions relies on a priori information of system model.

Most *model-based* observers are limited to a specific type of disturbance and lack universality. For disturbances with a bounded variation rate, the disturbance estimation error can converge to a bounded set [24], [25]. The upper bound of the set can be regulated by observer gains, which involves the trade-off between estimation accuracy and noise amplification.

2) *Data-Driven-Based Methods*: Benefiting from improvement in computing power and learning algorithms, *data-driven-based* methods appear to be another option for handling uncertainty. There are two general classes: full dynamics learning [26], [27] and partial dynamics learning [28]. We mainly focus on the latter where partially known dynamics can be utilized. In [29], a stochastic regime is designed to capture the uncertainty and reproduce the variability observed in experiments, whereas the learned uncertainty has not been used in control. In [30], the *Gaussian* belief propagation strategy is employed to compute the uncertainty satisfying *Gaussian* distribution and subsequently tighten the constraints of model predictive control (MPC). In [31], a hierarchical framework is proposed by merging traditional feedback control and residual reinforcement learning (RL) for contact-intensive tasks. In a similar manner,

the hierarchical framework is adopted in [32], where the additive unknown dynamics is approximated by *Gaussian* processes. Although this hierarchy scheme can learn the residual model with fewer samples and rollouts compared to pure RL, the learning procedure is still time-consuming. For example, three hours are needed for the block assembly task [31], and 15 s are needed for variable stiffness actuator pendulum swing-up [32]. In addition, one cannot ensure the outputs of the aforementioned learning methods are rigorously continuous, which is harmful to the actuators.

In summary, with respect to *model-based* uncertainty observers, abrupt uncertainty can be handled promptly due to the rapid online adaptive features. However, the steady-state performance of most *model-based* observers is limited to a certain type of uncertainties and lacks generalization. By contrast, *data-driven-based* methods can achieve decent steady-state performance, whereas the transient antidisturbance performance (learning phase) is usually unsatisfactory, as the construction procedure of the initial training dataset takes time. A random policy is frequently adopted at the first stage of learning [27], which may be dangerous for robots (e.g., underactuated aerial robots). To obtain the rapid transient reaction ability of *model-based* methods and high-precision steady-state tracking performance of *data-driven-based* methods simultaneously, a preliminary attempt combines the two methods by directly superimposing the outputs of both [33]. However, such schemes have not handled the discontinuity of learning methods well and the two methods may contradict when estimating uncertainties with similar features.

III. PROBLEM FORMULATION

Consider an affine nonlinear control system as

$$\dot{x} = f_x(x) + f_u(x)u + \Delta \quad (1)$$

where $x \in \mathbb{R}^n$ and $u \in \mathbb{R}^m$ denote the state and the control input, respectively; $f_x(\cdot) \in \mathbb{R}^n$ and $f_u(\cdot) \in \mathbb{R}^{n \times m}$ are smooth nonlinear mappings and continuously differentiable; $f_u(\cdot)$ is of full row rank to ensure uncertainty compensability; $\Delta \in \mathbb{R}^n$ represents the unknown portion, including external disturbance and internal model uncertainty. The compensation of Δ is assumed within the capability of the control input.

A general type of uncertainty is considered in this work

$$\Delta = h(\Delta, x, \ell_k, \ell_u) \quad (2)$$

where $h(\cdot) \in \mathbb{R}^n$ is an unknown nonlinear mapping to be approximated by available input and output data; $\ell_k \in \mathbb{R}^k$ and $\ell_u \in \mathbb{R}^u$ denote other known and unknown external disturbance vectors, respectively. For example, in the case of the quadrotor (Section VI-C), Δ represents the wind disturbance, which depends on the system state and external wind speed [34], [35], [36], where ℓ_u can represent the external wind speed that cannot be obtained. Unlike x and ℓ_k , ℓ_u cannot be regarded as a learning feature to fit the uncertainty model (2) in the online learning paradigm. In this work, an alternative strategy is designed in Section IV-E to attenuate the influence of ℓ_u . Let $s = n + k + u$.

Denote

$$z = [x^T, \ell_k^T, \ell_u^T]^T \in \mathbb{R}^s \quad (3)$$

Assumption 1: (Uncertainty Rate Boundness [16]) There exists an unknown positive value $\bar{\Delta}$, such that $\|\dot{\Delta}\| \leq \bar{\Delta}$.

The reasonability and limitation of the above assumption are analyzed in Appendix A.

Problem Statement: Consider system (1). The objective is to develop an evolutionary *model-based* uncertainty observer to deal with uncertainty (2). By resorting to the observer, the rapid transient reaction performance and the high-precision steady-state tracking performance are required to be achieved.

Overall Solution: Fig. 2 presents the proposed EVOLVER framework. In the transient state, an initial uncertainty observer is designed to handle the unknown uncertainty, and the initial training dataset is constructed meanwhile. In the steady state, the Koopman operator is leveraged to explore the unknown model of internal and external uncertainties, which is subsequently utilized in the uncertainty observer. The whole framework features a provable convergence guarantee in optimal conditions. The modules constituting Fig. 2 are detailed in followed sections.

IV. THEORETICAL FRAMEWORK

In this section, the EVOLVER is developed to handle the unmodeled uncertainty of (2).

A. Initial Uncertainty Observer

To maintain the rapid transient reaction performance, the initial uncertainty observer is designed. The objective of the nonlinear uncertainty observer is to achieve

$$\dot{\tilde{\Delta}} = L\tilde{\Delta} \quad (4)$$

where the symmetric matrix $L \in \mathbb{R}^{n \times n}$ denotes the observer gain matrix. The disturbance estimation error $\tilde{\Delta}$ converges exponentially to zero, as all of the eigenvalues of L have negative real parts.

From (1) and (4), the dynamics of the disturbance estimate $\hat{\Delta}$ can be derived as follows:

$$\begin{aligned} \dot{\hat{\Delta}} &= \dot{\Delta} - L(\Delta - \hat{\Delta}) \\ &= \dot{\Delta} - L(\dot{x} - f_x(x) - f_u(x)u - \hat{\Delta}). \end{aligned} \quad (5)$$

Note that $\tilde{\Delta} = \Delta - \hat{\Delta}$. However, in most robotic applications $\dot{x}$ is unknown or noise-filled. To circumvent this issue, define an auxiliary function $p(x) = Lx$ and an auxiliary variable $\xi = \dot{\Delta} + p(x)$. Without considering $\dot{\Delta}$, the initial nonlinear uncertainty observer is formalized as

$$\begin{cases} \dot{\xi} = L(f_x(x) + f_u(x)u + \hat{\Delta}) \\ \dot{\hat{\Delta}} = \xi - p(x). \end{cases} \quad (6)$$

Lemma 1. (Initial Observer Convergence): Consider the nonlinear system (1) with the uncertainty (2). Under Assumption 1, the estimation error of the initial nonlinear uncertainty observer (6) can exponentially enter into a bounded set, if $\lambda_M(L) < -\frac{1}{4\epsilon}$

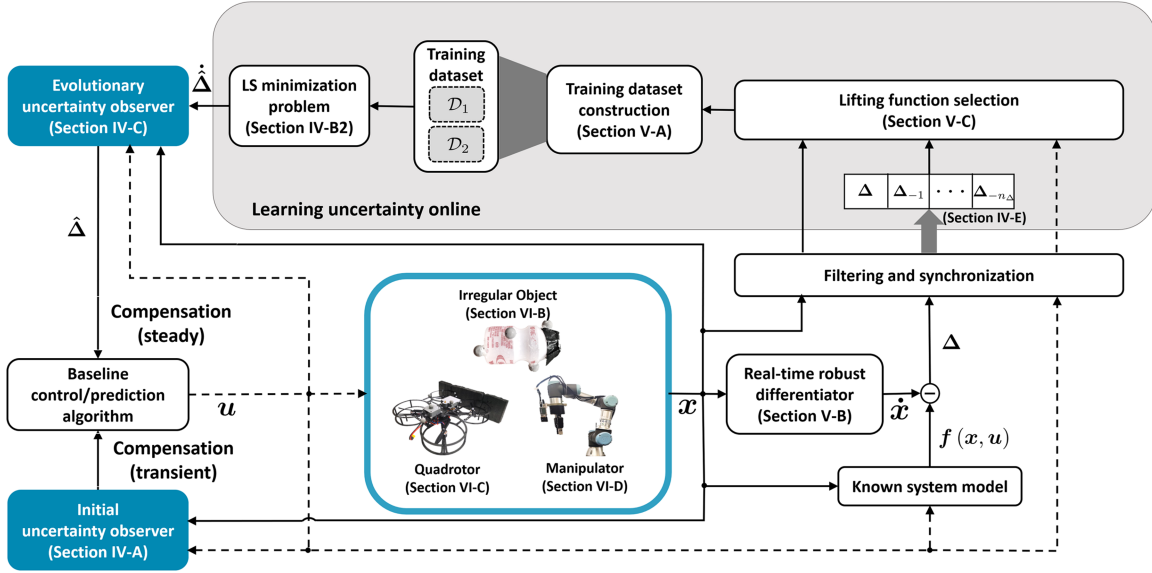


Fig. 2. Overall framework of our proposed EVOLVER. In the trajectory prediction example, the signal flow denoted by the dotted line does not exist.

is satisfied where ϵ is an arbitrary positive number related to the bounded set.

Proof: See Appendix B. ■

The uncertainty observer (6) is first proposed in [20] for constant disturbance, i.e., $\dot{\Delta} = 0$. The convergence analysis under Assumption 1 is further extended here. The detailed steps of proof in Appendix B will recur in the rest of this article (i.e., Lemma 3 and Theorem 1). The strength of (6) is the exponential convergence property in transient state, even for the unknown uncertainty.

B. Learning Uncertainty Model With Koopman Operator

The uncertainty model $\dot{\Delta}$ is not included in the initial observer (6). To achieve high-precision steady-state estimation performance, a learning approach is needed to explore the uncertainty model. In order to learn the uncertainty model, an easy-to-implement and explicit *data-driven-based* method is required in our work. Easy-to-implement requires the chosen method to be executed online. Explicit model expression, instead of a black-box input–output mapping, is demanded due to the learned structure with sound interpretability used in the evolutionary uncertainty observer. Koopman operator [37] is such a method to meet these requirements.

1) *Koopman Operator:* In the Koopman operator framework [37], the behavior of a nonlinear system can be captured by an infinite dimensional linear operator, thereby allowing the use of powerful and comprehensive analysis techniques for linear systems. This feature enables a wide variety of applications, such as nonlinear observability analysis [38], state observer design [39], [40], and shared control [41]. In [42] and [43], researchers extend the Koopman operator to dynamical systems with exogenous inputs. For ease of exposition, we recall the Koopman operator without considering the factor z in (3) for

the time being, and subsequently describe how to modify the algorithm to ensure its integrity.

Consider the uncertainty model

$$\dot{\Delta} = h(\Delta). \quad (7)$$

Define the lifting function $\varphi(\Delta) : \mathbb{R}^n \rightarrow \mathbb{R}$, which belongs to an infinite-dimensional function space $\mathcal{F}$. Then, the Koopman operator $\mathcal{K} : \mathcal{F} \rightarrow \mathcal{F}$ is defined as [37]

$$\mathcal{K}\varphi(\Delta(t_0)) = \varphi \circ \Delta(t_0 + t) \quad (8)$$

where t_0 represents the initial moment and $\Delta(t_0 + t) = \Delta(t_0) + \int_{t_0}^{t_0+t} h(\Delta)dt$.

Due to the infinite-dimensional feature of the Koopman operator $\mathcal{K}$, the direct implementation is impossible. Extended dynamic mode decomposition (EDMD) [44] offers a data-driven option of approximating $\mathcal{K}$ to a finite-dimensional transfer matrix by a series of nonlinear lifting functions.

2) *EDMD for Uncertainty System:* The observer functions of the dynamic mode decomposition [45] depend on the linear measurements of system states. EDMD can capture nonlinear transient behavior. By combining various linear control techniques, EDMD greatly promotes the application of the Koopman operator in robotics [46], [47], [48]. In this study, the EDMD is applied to the nonlinear uncertainty system (7).

To begin with, define a set of linearly independent lifting functions as

$$\Phi(\Delta) := [\Delta^T, \varphi_1(\Delta), \dots, \varphi_{N-n}(\Delta)]^T \in \mathbb{R}^N \quad (9)$$

where $\varphi_i(\cdot) \in \mathcal{F}$ and N represents the number of lifting functions. For the convenience of extracting original uncertainty from lifting functions, the first n functions are selected as the uncertainty Δ . Define a subspace $\mathcal{F}_N \subset \mathcal{F}$, which is spanned by the lifting functions in $\Phi(\Delta)$, i.e., $\{\Delta^T, \varphi_1, \dots, \varphi_{N-n}\}$.

The EDMD constructs a finite-dimensional approximation $K_{NM} \in \mathbb{R}^{N \times N} : \mathcal{F}_N \rightarrow \mathcal{F}_N$ of the Koopman operator $\mathcal{K}$ by

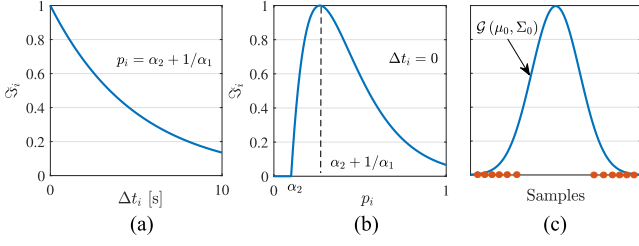


Fig. 3. Illustration of information value $\mathfrak{I}_i$. (a) *Timeliness*. The information value $\mathfrak{I}_i$ is discounted over time with a fixed *correlation*. (b) *Correlation*. When $p_i > \alpha_2 + \frac{1}{\alpha_1}$, the sample S_i becomes more and more valuable with probability p_i decreasing. When $p_i < \alpha_2 + \frac{1}{\alpha_1}$, the lower the probability p_i , the more likely S_i is to be regarded as the outlier. (c) *Marginalization* phenomenon (not uniform distribution) when employing only one multivariate *Gaussian* distribution.

solving the least-squares (LS) minimization problem

$$\min_{\mathbf{K}_{NM}} \frac{1}{2} \sum_{j=1}^M \left\| \Phi_{t_s}^j - \mathbf{K}_{NM} \Phi^j \right\|^2 \quad (10)$$

where M represents the samples number, Φ^j is the lifting function corresponding to the j th sample Δ^j ($j = 1, 2, \dots, M$), t_s denotes the sampling time, and $\Phi_{t_s}^j$ is the evolution of Φ^j after one sampling time.

Fortunately, the problem (10) has the closed-form solution

$$\mathbf{K}_{NM} = \frac{1}{M} \sum_{j=1}^M \Phi_{t_s}^j (\Phi^j)^T \left[\frac{1}{M} \sum_{j=1}^M \Phi^j (\Phi^j)^T \right]^\dagger. \quad (11)$$

The linearized model of (7) in its lifting space $\mathcal{F}_N$ can be obtained

$$\Phi(\Delta(t_0 + t_s)) = \mathbf{K}_{NM} \Phi(\Delta) + \mathbf{r}(t_0) \quad (12)$$

where $\mathbf{r} \in \mathbb{R}^N$ is the residual error function, by considering that $\mathcal{F}_N$ may not be an invariant subspace of $\mathcal{F}$. If the selected finite lifting functions do form an invariant subspace, the resulting linear model can fully describe the system behavior, i.e., $\mathbf{r}(t_0) = 0$ [49].

The continuous-time dynamics (7) requires reformulation as the discrete-time function (12). The approximate continuous differential equation can be obtained

$$\begin{cases} \dot{\Phi}(\Delta) = \mathbf{A}_\Delta \Phi(\Delta) + \mathbf{r}_1 \\ \Delta = \mathbf{C} \Phi(\Delta) \end{cases} \quad (13)$$

where $\mathbf{r}_1 \in \mathbb{R}^N$ denotes the residual function error

$$\mathbf{A}_\Delta = \log(\mathbf{K}_{NM}) / t_s \in \mathbb{R}^{N \times N} \quad (14)$$

with natural logarithm function $\log(\cdot)$, and $\mathbf{C} = [\mathbf{I}_{n \times n}, \mathbf{0}_{n \times (N-n)}] \in \mathbb{R}^{n \times N}$. The principal matrix logarithm $\log(\cdot)$ demands all of the eigenvalues of the matrix have nonnegative real parts, which may fail when the dataset is insufficient in practice. Without increasing more system measurements like [48],

the designed dataset construction method in Section V-A can effectively handle this issue, as shown in Fig. 6(c) and (d).

Lemma 2. (EDMD Convergence [50]): The sequence of operators $\mathbf{K}_{NM}$ converges to $\mathcal{K}$ as $M \rightarrow \infty$ and $N \rightarrow \infty$, if the following optimal conditions are satisfied.

- i) The lifting functions are selected from an orthonormal basis of $\mathcal{K}$.
- ii) The samples are drawn independently.
- iii) The Koopman operator $\mathcal{K}$ is bounded.

The proof procedure of Lemma 2 is detailed in [50] and [51]. The descriptions of the conditions are rephrased for ease of understanding. If condition (i) is satisfied, the positive measure μ of Δ to make $\mathbf{c}^H \Phi = 0$ held is zero. $\mathbf{c} \in \mathbb{C}^N$ denotes any nonzero vector and $\mathbf{c}^H$ denotes the Hermitian transpose of $\mathbf{c}$. Conditions (i) and (ii) lead to that $\mathbf{K}_{NM}$ converges to $\mathcal{K}_N$ as $M \rightarrow \infty$ [51], where $\mathcal{K}_N$ is the $L_2(\mu)$ -orthogonal projection of the Koopman operator $\mathcal{K}$ on the subspace $\mathcal{F}_N$. Condition (i) can be satisfied using Hermite polynomial-based lifting functions [52]. Conditions (i) and (iii) result in that $\mathbf{K}_N$ converges to $\mathcal{K}$ as $N \rightarrow \infty$ [50]. Condition (iii) can be satisfied if the learned system is invertible, Lipschitz with Lipschitz inverse, and the measure is absolutely continuous.

Take the factor $\mathbf{z}$ of (3) into account, which contributes to the evolution of Δ . This can be treated in an analogous way by expanding the states $\Phi(\Delta)$. Following [53] and [46], design additional lifting functions $\Psi(\Delta, \mathbf{z}) : \mathbb{R}^n \times \mathbb{R}^s \rightarrow \mathbb{R}^{N_2}$, which take the coupling terms of Δ and $\mathbf{z}$ as variables. By concatenating with the previous lifting functions $\Phi(\Delta)$ as $\mathbf{S}(\Delta, \mathbf{z}) := [\Phi(\Delta)^T, \Psi(\Delta, \mathbf{z})^T]^T$, it can be rendered

$$\dot{\mathbf{S}}(\Delta, \mathbf{z}) = \mathbf{A} \mathbf{S}(\Delta, \mathbf{z}) + \mathbf{r}_2 \quad (15)$$

with $\mathbf{A} \in \mathbb{R}^{(N+N_2) \times (N+N_2)}$ and residual error $\mathbf{r}_2 \in \mathbb{R}^{(N+N_2)}$. Similar to (10)–(14), the Koopman operator $\mathbf{A}$ in continuous differential equation can be obtained and partitioned as

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_\Delta & \mathbf{A}_z \\ * & * \end{bmatrix} \quad (16)$$

where $\mathbf{A}_\Delta \in \mathbb{R}^{N \times N}$ and $\mathbf{A}_z \in \mathbb{R}^{N \times N_2}$. Focusing on the evolution of $\Phi(\Delta)$, it follows that:

$$\begin{cases} \dot{\Phi}(\Delta) = \mathbf{A}_\Delta \Phi(\Delta) + \mathbf{A}_z \Psi(\Delta, \mathbf{z}) + \mathbf{r}_3 \\ \Delta = \mathbf{C} \Phi(\Delta) \end{cases} \quad (17)$$

where $\mathbf{r}_3 \in \mathbb{R}^N$ is the first N rows of $\mathbf{r}_2$ in (15).

In practice, the residual $\mathbf{r}_3$ is mainly caused by the unknown factor ℓ_u , the monotony of the dataset, data noise, and improper choice of lifting functions. The strategies for reducing $\mathbf{r}_3$ are provided in Section IV-E and Section V.

Based on the above analysis, a corollary of the uncertainty learnability (i.e., $\mathbf{r}_3 = 0$) can be obtained.

Corollary 1. (Uncertainty Learnability): The uncertainty Δ described in (2) is fully learnable by the EDMD, i.e., $\mathbf{r}_3 = 0$, if the following conditions are satisfied.

- i) Δ does not depend on ℓ_u .
- ii) The optimal conditions of Lemma 2 are satisfied.

Until now, the procedure of learning the uncertainty model (2) by virtue of the Koopman operator has been presented. Direct

employment of the learned model (17) to estimate uncertainty faces two challenges: how to guarantee the transient performance while the initial training dataset is being constructed and how to obtain the initial estimation of uncertainty. Instead of directly employing the learned model (17), a *model-based* uncertainty observer, taking advantage of the learned model, intends to overcome the challenges in the next section.

C. Evolutionary Uncertainty Observer

If the uncertainty model (17) has been learned, (5) can be adjusted to

$$\begin{aligned}\dot{\hat{\Delta}} &= \dot{\Delta} - L(\Delta - \hat{\Delta}) \\ &= CA_{\Delta}\Phi + CA_z\Psi \\ &\quad - L(\dot{x} - f_x(x) - f_u(x)u - \hat{\Delta}).\end{aligned}\quad (18)$$

Therefore, the nonlinear uncertainty observer is formalized as

$$\begin{cases} \dot{\xi} = CA_{\Delta}\Phi + CA_z\Psi + L(f_x(x) + f_u(x)u + \hat{\Delta}) \\ \hat{\Delta} = \xi - p(x). \end{cases}\quad (19)$$

Lemma 3. (Observer Convergence): Consider the nonlinear system (1) with the uncertainty (2). Suppose the conditions in Lemma 2 are all satisfied and ℓ_u is neglected. The estimation error of the designed nonlinear uncertainty observer (19) can exponentially go to zero regardless of x and u , if L is negative definite.

Proof: See Appendix C. ■

Notice that the calculations of $\Phi(\Delta)$ and $\Psi(\Delta, z)$ in (19) need current uncertainty Δ . Because accurate $\dot{x}$ requires good differentiating and filtering tools, which induce non-negligible delays, Δ cannot be obtained precisely from the current state by using $\Delta = \dot{x} - f_x(x) - f_u(x)u$. Nevertheless, it can be obtained by prediction from previous accurate fitting data with the learned uncertainty model. Moreover, even though the learned part ($CA_{\Delta}\Phi + CA_z\Psi$) is discontinuous with the update of A_{Δ} and A_z , the output of the uncertainty observer (19) is still continuous by virtue of its integral feature.

D. Overall Theoretical Framework

Regard $\mathcal{D} = \{S^i(\Delta, z) \mid i = 1, 2, \dots, M+1\}$ as the training dataset with $M+1$ samples, where $S^i(\Delta, z)$ represents the sample of i th sampling period before the current time. Then, the dataset tuple $\mathcal{D}_1 = \{S^i(\Delta, z) \mid i = 2, 3, \dots, M+1\}$ and $\mathcal{D}_2 = \{S^i(\Delta, z) \mid i = 1, 2, \dots, M\}$ are constructed from $\mathcal{D}$, which implies that $\mathcal{D}_1$ is the evolution of $\mathcal{D}_2$ after one sample period.

Algorithm 1 illustrates the overall online learning, prediction, and compensation procedure. In some cases, an autodetective mechanism is needed to judge the presence of the uncertainty, such that the EVOLVER can be activated. It can be achieved by enabling the initial uncertainty observer (6) all the time without compensation, and detecting if the average estimated

Algorithm 1: EVOLVER Framework.

Initialize: Observer gain L ; lifting functions S ; step size t_s ; $j \leftarrow 0$; threshold T_1 and T_2 ; internal clocking $t_{in} \leftarrow 0$;
Repeat: Detect the presence of uncertainty;
1: **for** $i = 0$ to ∞
2: **if** $i \cdot t_s \leq T_1$
3: Execute the initial observer (6);
4: Get current sample (Δ, z) and lift it to $S^i(\Delta, z)$;
5: Add last $S^{i-1}(\Delta, z)$ and current $S^i(\Delta, z)$ to datasets $\mathcal{D}_1$ and $\mathcal{D}_2$, respectively;
6: **return** $\hat{\Delta}$;
7: **else**
8: Use dataset $\mathcal{D}_1$ and $\mathcal{D}_2$ to fit model A by (11);
9: $t_{in} \leftarrow t_{in} + t_s$;
10: **if** $j == 0$ **then**
11: $j \leftarrow j + 1$;
12: $A_j \leftarrow A$;
13: **else**
14: **if** $t_{in} > T_2$ **then**
15: $j \leftarrow j + 1$;
16: $A_j \leftarrow A$;
17: $t_{in} \leftarrow 0$;
18: **end if**
19: **end if**
20: Execute the observer (19) with updated A_j ;
21: **return** $\hat{\Delta}$;
22: Get current sample (Δ, z) and lift it to $S^i(\Delta, z)$;
23: Add last $S^{i-1}(\Delta, z)$ and current $S^i(\Delta, z)$ to datasets $\mathcal{D}_1$ and $\mathcal{D}_2$, respectively;
24: **end if**
25: Compensate $\hat{\Delta}$ by the baseline controller.
26: **end for**

uncertainty over a period of time exceeds a certain threshold. See supplementary materials for details. The EVOLVER mainly consists of two stages: the transient phase (lines 3–6) and the steady-state phase (lines 8–23). In the transient phase, the initial uncertainty observer (6) works while the initial training dataset is constructed. After $M+1$ samples are collected (line 2), the system enters the steady-state phase. In the steady-state phase, the EDMD is used to fit the uncertainty model and subsequently the evolutionary uncertainty observer (19) takes over. Especially, in order to accurately capture the uncertainty of interest (i.e., the uncertainty around the current operating point) with system (1) evolving, the learned uncertainty model is refreshed in an inherent renewal cycle T_2 or by a difference judgment (lines 14–18). Note that the switch from the initial uncertainty observer (6) to the evolutionary uncertainty observer (19) is continuous, which is benefited from the integral feature of the uncertainty observer.

Theorem 1. (Convergence of Algorithm 1): For nonlinear system (1) with unmodeled uncertainty (2), under Algorithm 1, if the conditions of Lemma 2 are all satisfied and ℓ_u is neglected, the uncertainty estimation error $\tilde{\Delta}$ in Algorithm 1 converges

to zero asymptotically. Furthermore, if ℓ_u is considered, $\tilde{\Delta}$ will converge to an arbitrarily small bounded set by decreasing $\lambda_M(\mathbf{L})$.

Proof: See Appendix D. ■

The EVOLVER framework is able to extend the application range of the normal disturbance observer (DO) which is first proposed in [20] and is intended for dealing with the constant disturbance only. The presented scheme in Theorem 1 can handle both time-varying unknown dynamics and environmental disturbances.

E. Attenuating the Unpredictable Uncertainty

As stated in Theorem 1, the zero convergence error neglects the influence of ℓ_u . This effect can be attenuated by decreasing $\lambda_M(\mathbf{L})$. However, the effectiveness of such an approach is limited, because noise and actuator delay in real robotics limit the magnitude of $\mathbf{L}$. Hence, further solutions for attenuating the uncertainty caused by ℓ_u are needed.

Due to the lack of relevant knowledge, it is intractable to completely predict the impact of ℓ_u . Before proceeding, it is assumed that the motion exhibits specific autocorrelation, such that it is amenable to forecasting. We further mine the historical data to find valuable patterns by regarding

$$\ell_u = [\Delta_{-1}^T, \Delta_{-2}^T, \dots, \Delta_{-n_\Delta}^T]^T \quad (20)$$

where Δ_{-i} ($i \in \{1, 2, \dots, n_\Delta\}$) denotes the historical uncertainty data of the past i sampling cycles, and n_Δ represents the chosen number of historical samples. The guideline for choosing n_Δ in real applications is provided in supplementary materials.

To achieve disturbance prediction, in [14] and [54], the disturbance in each dimension is parameterized with historical states linearly. The coefficients are optimized using LS. In such a way, the continuity of the predicted uncertainty cannot be guaranteed, which may induce a chattering phenomenon [see Fig. 10(g)]. In the EVOLVER framework, this drawback is avoided as a result of the natural integration of the uncertainty observer (19).

Note that for some cases, the evolution of Δ depends on the higher derivative of the system state, which cannot be measured. One way is to treat it as ℓ_u and apply the above attenuation strategy. Another way is to utilize the following RD (22) to obtain the higher derivative with accurate and robust performance.

V. PRACTICAL CONSIDERATION

Without a doubt, the requirements $M \rightarrow \infty$ and $N \rightarrow \infty$ in Lemma 2 are unreachable in practice. The practical problems, including the information richness in the training dataset, data noise, and lifting function selection are analyzed. The corresponding solutions are designed to pursue optimality.

A. Dataset Construction

As stated in Lemma 2, zero-error learning demands infinite independent samples. Moreover, the calculation of the principal matrix logarithm in (14) demands a sufficient dataset. However, the EDMD framework is executed over a given sliding-time window of limited historical state data, in which the data may be

strongly correlated. In particular, the latest dataset can be more monotonous as the robot operates at a fixed point. Very few works take this point into account. In [46], an active learning framework is proposed to learn the Koopman operator by driving the robot to explore informative state space. By contrast, the original robotics task is expected unchanged in this study. We next construct a diverse dataset to cover more states with valuable information by evaluating each new sample.

Define a vector $\mathfrak{S} \in \mathbb{R}^M$, which contains the evaluated values [defined in (21) below] of all M samples of $\mathcal{D}_2$ in the sequence. Denote $\mathfrak{S}_i$ as the i th sample $\mathbf{S}^i$ (abbreviation of $\mathbf{S}^i(\Delta, \mathbf{z})$) in $\mathcal{D}_2$ and its evaluated value, respectively. We assess the value of a sample by considering two factors: how new the sample is and how different the sample is from the others, referred to as *timeliness* and *correlation*, respectively. Focusing on *timeliness*, the value is discounted as time passes [i.e., the part $e^{-\beta|\Delta t_i|}$ in (21)]. With respect to *correlation*, a probabilistic technique is utilized [i.e., the part $\mathfrak{f}(p_i)$ in (21)]. The previous dataset $\mathcal{D}_2$ is fitted by a multivariate *Gaussian* distribution $\mathcal{G}(\mu_0, \Sigma_0)$ first, and then the probability of each incoming sample is calculated. The lower the probability, the more valuable the new sample (i.e., including more additional information), and vice versa. To ease the computational burden, the fitted variables can choose a few principal ones. In addition, the outlier problem needs to be considered as well [55], which may have a very low probability in $\mathcal{G}(\mu_0, \Sigma_0)$. By combing the foregoing *timeliness* and *correlation*, the evaluation index is formalized

$$\mathfrak{S}_i = \mathfrak{f}(p_i) e^{-\beta|\Delta t_i|} \quad (21)$$

where e is the natural constant, $\beta > 0$ is a time discount factor, Δt_i is the time difference from the current time to the i th sampling time, p_i represents the probability of i th sample in the multivariate *Gaussian* distribution, denoted by $p_i = \mathcal{G}(\mathbf{S}_i)$, and $\mathfrak{f}(p_i)$ represents a nonlinear mapping defined as $\mathfrak{f}(p_i) = \frac{\alpha_1(p_i - \alpha_2)}{e^{\alpha_1(p_i - \alpha_2)}} e$ with positive parameters α_1 and α_2 . The *correlation* degree and outlier rejection can be adjusted by α_1 and α_2 , respectively. Fig. 3(a) and (b) illustrate the characteristics of the resulting index. Given the information value of each sample, the sample with the lowest value in the dataset is replaced by the incoming one with a higher value.

Experiments reveal that using one multivariate *Gaussian* distribution may lead to *marginalization*, as depicted in Fig. 3(c). To avoid this issue, in the process of *Gaussian* distribution fitting, we can shuffle the data in $\mathcal{D}_2$, divide $\mathcal{D}_2$ into n_G segments, and fit them with n_G ($n_G > 1$) *Gaussian* distributions (denoted as $\{\mathcal{G}_i(\mu_i, \Sigma_i)\}_{i=1}^{n_G}$) in the sequence.

The dataset construction flow is summarized in Algorithm 2. The lowest evaluated value and its corresponding sample in $\mathcal{D}_2$ are denoted by $\mathfrak{S}_m$ and $\mathbf{S}_m$, whereas $\mathbf{S}_n$ represents the new incoming sample and $\mathcal{G}_m(\mathbf{S}_n)$ represents the maximum probability of $\mathbf{S}_n$ in $\{\mathcal{G}_i(\mu_i, \Sigma_i)\}_{i=1}^{n_G}$. As $t > T_1$, $\mathbf{S}_n$ will be evaluated by (21) with $\Delta t = 0$ (lines 10–12), denoted as $\mathfrak{S}_n$. If $\mathfrak{S}_n > \mathfrak{S}_m$, $\mathbf{S}_n$ will replace $\mathbf{S}_m$ (lines 14), otherwise, $\mathbf{S}_n$ is discarded (line 16). Notice that for saving computing resources, the refresh cycle of μ_i and Σ_i is decided by T_3 (lines 3–8). The efficiency of our designed dataset construction method is

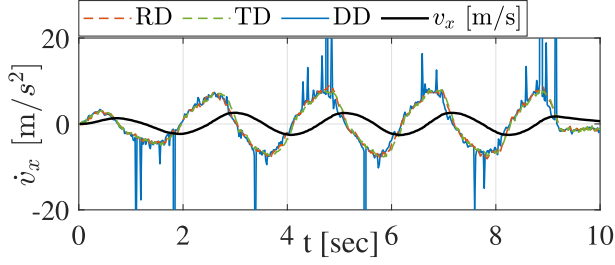


Fig. 4. Performance of the employed differentiator. The solid black curve represents the real flight data of the quadrotor velocity that contains velocities between 0 to 3 m/s. The performance of three kinds of differentiators: RD (22), TD [16], and DD (23) are shown in dashed red, dashed green, and solid blue curves, respectively. The parameters of the robust one are set as $k_1 = 16.97$, $k_2 = 36$, $k_3 = 1.48$.

demonstrated in Section VI-B, as shown in Fig. 6(c). It can be seen that after considering the information index $\mathfrak{S}$, the dataset covers more states with valuable information. One limitation of this strategy is the lack of rigorous theoretical guarantees. Future work will pursue improving this dataset construction method from a more theoretical aspect.

Algorithm 2: Dataset Construction Flow.

Initialize: Construct initial $\mathcal{D}$ as $t \leq T_1$ and fit initial μ_0, Σ_0 ; threshold T_3 ; internal clocking $t_i \leftarrow 0$;

- 1: **while** $t > T_1$ **do**
- 2: $t_{in} \leftarrow 0$;
- 3: **if** $t_{in} > T_3$ **then**
- 4: $t_{in} \leftarrow 0$;
- 5: Reorganize newest $\mathcal{D}_1$ and $\mathcal{D}_2$;
- 6: Refresh $\{\mathcal{G}_i(\mu_i, \Sigma_i)\}_1^{n_g}$ with $\mathcal{D}_2$;
- 7: Refresh $\mathfrak{S}$ by (21);
- 8: **end if**
- 9: Find $\mathfrak{S}_m, S_m$;
- 10: Get the new incoming sample S_n ;
- 11: Calculate the probability $p_n = \mathcal{G}_m(S_n)$;
- 12: Calculate $\mathfrak{S}_n$ by (21) with $\Delta t = 0$;
- 13: **if** $\mathfrak{S}_n > \mathfrak{S}_m$ **then**
- 14: Replace $\mathfrak{S}_m$ by $\mathfrak{S}_n$; replace S_m by S_n in $\mathcal{D}_2$;
- 15: replace corresponding sampling in $\mathcal{D}_1$;
- 16: **else**
- 17: Discard S_n ;
- 18: **end if**
- 19: **return** $\mathcal{D}$.
- 20: **end while**

B. Data Noise Handling

The dataset of snapshot pairs in the EDMD is constructed from noisy states. In order to save computing resources, a lightweight processing approach is adopted in this work. At the data differential stage, the derivative of a signal $s \in \mathbb{R}$ is obtained

by a real-time RD [56], which is formalized as

$$\begin{cases} \dot{y}_1 = k_1 \nu_1(s - y_1) + y_2 \\ \dot{y}_2 = k_2 \nu_2(s - y_1) \\ \dot{s} = y_2 \end{cases} \quad (22)$$

with auxiliary variables $y_1 \in \mathbb{R}$, $y_2 \in \mathbb{R}$; auxiliary functions $\nu_1(x) = k_3^{-1} \Theta(k_3^2 x) \in \mathbb{R}$, $\nu_2(x) = 2\Theta(k_3^2 x) \Theta(k_3^2 x)' \in \mathbb{R}$; and tuning parameters $k_1 \in \mathbb{R}$, $k_2 \in \mathbb{R}$, and $k_3 \in \mathbb{R}$. $\Theta(\cdot)$ is called the differentiator generating function and $\Theta(\cdot) = \text{sign}(\cdot)(|\cdot|^{\frac{1}{2}} + |\cdot|^{\frac{3}{2}})$ is chosen. For a signal with a bounded second derivative, the differentiator can converge to the derivative of such a signal within a predefined bounded time, which is robust to measurement noise.

Fig. 4 shows the performance of the RD (22) with the real velocity data of the quadrotor used in Section VI-C. The discrete differentiator (DD) is formalized as

$$\dot{s} = \frac{s - s_{-3t_s}}{3t_s} \quad (23)$$

with sampling time t_s . s_{-3t_s} denotes the historical s before 3 sampling cycles. Although the DD has been smoothed by averaging the last three differences, the obtained $\dot{s}$ contains serious noise. In sharp contrast, the employed RD shows accurate and robust performance.

Moreover, the performance of tracking differentiator (TD) proposed by [16] is also shown in Fig. 4. Its performance can reach the level of the RD after careful parameter adjustment. Due to the provable convergence guarantee and clear guidance on parameter adjustment [56], the RD is chosen in our work.

At the data filtering stage, the obtained $\dot{s}$ is filtered by a second-order low-pass filter. In this process, we can tolerate millisecond signal delay to some extent in order to achieve more accurate filtering.

C. Lifting Functions Selection

Another factor to be considered is the selection of lifting functions, which heavily influence the EDMD performance. In this work, there are two guidelines for choosing lifting functions. On the one hand, the underlying knowledge about robotics may provide a reasonable choice of lifting functions [57], e.g., the aerodynamic drag for an irregular free-flying object or a flying quadrotor [34], [35]. On the other hand, if the considered system structure is unknown, the lifting functions can be chosen by leveraging on Hermite polynomial [58] and Kronecker product to construct orthonormal basis [52] to satisfy the condition (i) of Lemma 2 (see Section VI-A).

VI. SIMULATION AND EXPERIMENTAL VALIDATIONS

In this section, one simulation and three experiments with different types of uncertainties are conducted. Notice that, with a little abuse of symbols, the adopted symbols in each experiment are independent.

Moreover, to facilitate quantitative analysis, mean absolute error (MAE) and root mean square error (RMSE) are defined as $MAE = \frac{1}{n_d} \sum_{i=1}^{n_d} \|x_i - x_{d,i}\|$, $RMSE =$

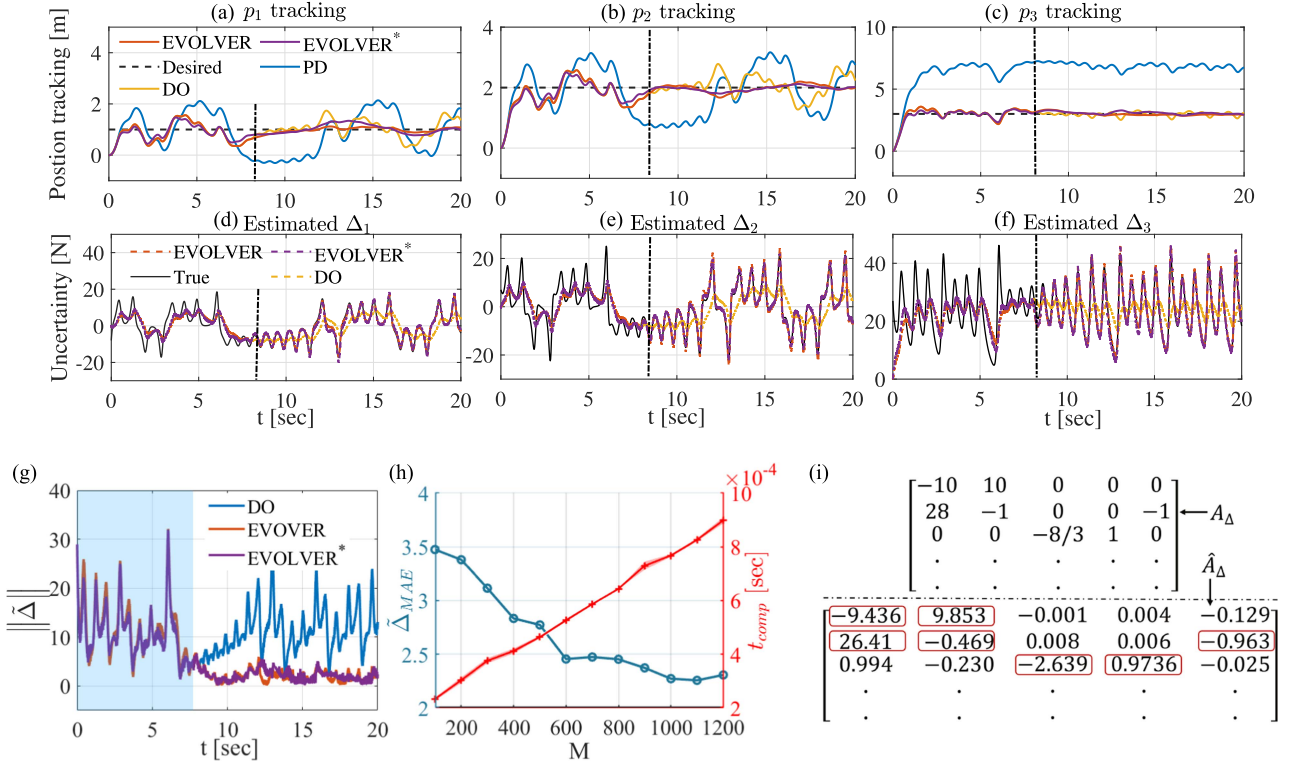


Fig. 5. Simulation results in the *Lorentz* example. (a)–(c) p tracking result. (d)–(f) The Δ estimation result. The transient and steady-state phases are separated by a black dotted line at $t = 8s$, i.e., the moment that the learned uncertainty model is first employed. The simulated results of four different schemes are shown in (a)–(f): the PD controller, the normal disturbance observer-based method [20], and the EVOLVER with manual and automatic constructed lifting functions, abbreviated to PD, DO, EVOLVER, and EVOLVER*, respectively, for simplicity. (g) Uncertainty estimation error $\|\hat{\Delta}\|$. (h) Computation time t_{comp} and uncertainty estimation error Δ_{MAE} under different sample number M . (i) The final estimated $\hat{A}_\Delta$ and true value A_Δ .

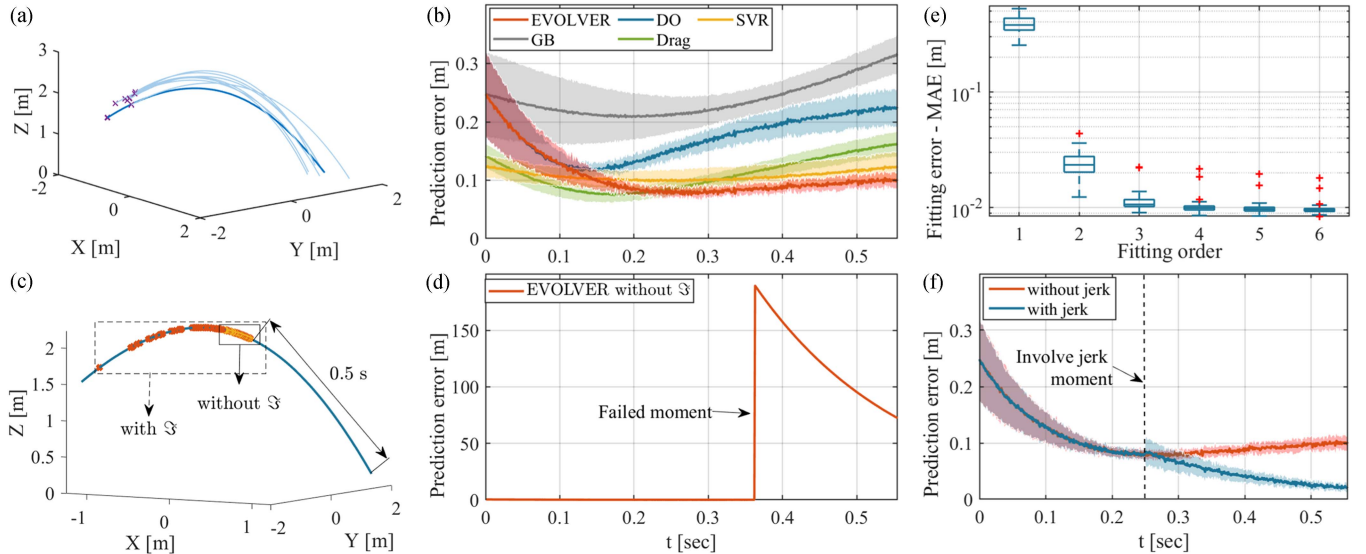


Fig. 6. Experimental results in the object trajectory prediction example. (a) All nine testing trajectories. The darkened one is utilized in (c) and (d) below. (b) Prediction errors of object trajectory with 0.5 s prediction horizon under different methods: proposed EVOLVER-based, gravity-based, normal disturbance observer-based [20], drag model-based [59], [60], [61], and SVR-based [62], [63], abbreviated to EVOLVER, GB, DO, Drag, and SVR, respectively, for simplicity. (c) Collected datasets in the final step with and without considering the information index Ξ are labeled on the test trajectory. (d) Failed convergence when Ξ is not considered. (e) Fitting errors of all recorded trajectories in different fitting orders. (f) Prediction errors with 0.5 s prediction horizon when the jerk information is involved.

$\sqrt{\frac{1}{n_d} \sum_{i=1}^{n_d} \|\mathbf{x}_i - \mathbf{x}_{d,i}\|^2}$, where n_d is the size of collected data, $\mathbf{x}_i$ and $\mathbf{x}_{d,i}$ are i th evaluated variable and its desired state.

A. Handling Chaotic Lorenz Uncertainty

The presented EVOLVER is applied to a second-order dynamical system with chaotic *Lorenz* uncertainty, which belongs to the type of $\dot{\Delta} = \mathbf{h}(\Delta)$.

1) *Problem Formulation*: The considered second-order *Newton* system is modeled as

$$\begin{cases} \dot{\mathbf{p}} = \mathbf{v}, \dot{\mathbf{v}} = \mathbf{a}, \\ m\mathbf{a} = \mathbf{u} + \Delta \end{cases} \quad (24)$$

with position $\mathbf{p} \in \mathbb{R}^3$, velocity $\mathbf{v} \in \mathbb{R}^3$ [i.e., $\mathbf{x}$ in (1)], acceleration $\mathbf{a} \in \mathbb{R}^3$, mass $m \in \mathbb{R}$, and control input $\mathbf{u} \in \mathbb{R}^3$. The uncertainty $\Delta = [\Delta_1, \Delta_2, \Delta_3]^T$ is generated by the following chaotic *Lorenz* system [57]

$$\begin{cases} \dot{\Delta}_1 = 10(\Delta_2 - \Delta_1) \\ \dot{\Delta}_2 = \Delta_1(28 - \Delta_3) - \Delta_2 \\ \dot{\Delta}_3 = \Delta_1\Delta_2 - \frac{8}{3}\Delta_3 \end{cases} \quad (25)$$

as shown in Fig. 5(d)–(f), by solid black curves. The reason for selecting the *Lorenz* uncertainty lies in its chaotic feature and rapid dynamic change, which can be very challenging to eliminate. From (25), it can be seen that the *Lorenz* uncertainty only depends on its own state, which belongs to the type of $\dot{\Delta} = \mathbf{h}(\Delta)$.

The objective is to design $\mathbf{u}$ so as to ensure that $\mathbf{p}$ and $\mathbf{v}$ track their desired states $\mathbf{p}_d$ and $\mathbf{v}_d$, respectively. The baseline controller adopts the proportional–derivative (PD) control, and the estimated uncertainty $\hat{\Delta}$ by our EVOLVER is compensated via feedforward. The controller is designed as

$$\mathbf{u} = \mathbf{K}_p \mathbf{e}_p + \mathbf{K}_v \mathbf{e}_v - \hat{\Delta} \quad (26)$$

with positive definite gain matrices $\mathbf{K}_p \in \mathbb{R}^{3 \times 3}$ and $\mathbf{K}_v \in \mathbb{R}^{3 \times 3}$, and tracking errors $\mathbf{e}_p = \mathbf{p}_d - \mathbf{p}$ and $\mathbf{e}_v = \mathbf{v}_d - \mathbf{v}$.

2) *Implementation Details*: Apart from our proposed EVOLVER being applied to estimate Δ , the normal disturbance observer-based method of [20] is chosen for comparison. For the sake of fairness, the gains $\mathbf{L}$ of both observers are set as the same. In addition, the measured $\mathbf{v}$ is corrupted by noise $\varepsilon_v \sim \mathcal{N}(\mathbf{0}, \sigma^2)$. The desired position and velocity are set as $[1, 2, 3]^T$ and $[0, 0, 0]^T$, respectively. The initial point of the *Lorenz* uncertainty is set as $[-8, 7, 27]^T$. All relevant simulation parameters are summarized in Table I. The involved scheme runs on a laptop with Intel(R) Core(TM) i7-8650 U CPU.

The lifting function is chosen as $\Phi(\Delta) = [\Delta_1, \Delta_2, \Delta_3, \Delta_1\Delta_2, \Delta_1\Delta_3]^T$ according to the structure of the *Lorenz* system (25). Moreover, the case when the structure of *Lorenz* system (25) is unknown in advance is also considered. In this case, the lifting function is constructed by Hermite polynomial and Kronecker product automatically. The fundamental variables are chosen as $\{\Delta_1, \Delta_2, \Delta_3\}$. By following the guideline of [52], the lifting function is constructed

TABLE I
EXPERIMENTAL PARAMETERS

	Parameters
Section IV-A	$\mathbf{K}_p = 6.25\mathbf{I}_{3 \times 3}, \mathbf{K}_v = 5\mathbf{I}_{3 \times 3}, \mathbf{L} = -3\mathbf{I}_{3 \times 3},$ $t_s = 0.01 \text{ s}, M = 700, T_1 = 8 \text{ s}, T_2 = 1 \text{ s},$ $\sigma = 0.1, k_1 = 60, k_2 = 450, k_3 = 4.182.$
Section IV-B	$\mathbf{L} = -5\mathbf{I}_{3 \times 3}, t_s = 0.001 \text{ s}, M = 100,$ $\beta = 0.2, \alpha_1 = 10, \alpha_2 = 0, n_G = 20,$ $T_1 = 0.05 \text{ s}, T_2 = 0 \text{ s}, \mathbf{D} = \text{diag}(0.1, 0.1, 0.096).$
Section IV-C	$\mathbf{L} = -0.4\mathbf{I}_{3 \times 3}, t_s = 0.01 \text{ s}, M = 300,$ $T_1 = 3 \text{ s}, T_2 = 0 \text{ s}, \mathbf{D} = \text{diag}(0.7, 0.31, 0),$ $n_\Delta = 2, k_1 = 16.97, k_2 = 36, k_3 = 1.48.$
Section IV-D	$\mathbf{K}_p = 8\mathbf{I}_{3 \times 3}, \mathbf{K}_i = 2\mathbf{I}_{3 \times 3}, \mathbf{L} = -3\mathbf{I}_{3 \times 3},$ $t_s = 0.08 \text{ s}, M = 200, T_1 = 1.6 \text{ s}, T_2 = 0 \text{ s},$ $\beta = 0.2, \alpha_1 = 10, \alpha_2 = 0, n_G = 10, n_\Delta = 5,$ $k_1 = \text{diag}(1.34, 1.34, 0.60),$ $k_2 = \text{diag}(0.23, 0.23, 0.05),$ $k_3 = \text{diag}(9.35, 9.35, 20.91).$

¹ Guidelines for adjusting these parameters are provided at <https://sites.google.com/view/buaa-evolver>.

as $\text{kron}(\text{kron}(\mathbf{H}(\Delta_1), \mathbf{H}(\Delta_2)), \mathbf{H}(\Delta_3))$, with zero- and first-order Hermite polynomials $\mathbf{H}(\cdot) = [H_0(\cdot), H_1(\cdot)]^T$ and Kronecker product $\text{kron}(\cdot, \cdot)$. Different from [52] employing probabilist's Hermite polynomial, the physicist's Hermite polynomial is adopted here.

3) *Simulation Results*: Fig. 5 shows the simulation results. From Fig. 5(a)–(c), it can be seen that the tracking performance of the normal disturbance observer-based method and EVOLVER is superior to the PD controlled one in transient phase ($t < 8\text{s}$), with less overshoot. In the event of p_3 tracking with huge initial uncertainty, the normal disturbance observer-based method and EVOLVER method can rapidly suppress the sudden uncertainty. However, the normal uncertainty observer can only capture the trend of *Lorenz* uncertainty roughly, as shown in Fig. 5(d)–(g). In the steady-state phase ($t > 8\text{s}$), after the learned uncertainty model is employed, the EVOLVER can accurately predict the small and rapid variations of the *Lorenz* uncertainty, with either manually or automatically chosen lifting functions. The final learned $\hat{\mathbf{A}}_\Delta$ (denoted as $\hat{\mathbf{A}}_\Delta$) and true $\mathbf{A}_\Delta$ in (13) are shown in Fig. 5(i), and it can be seen that those important nonzero elements are well captured.

Fig. 5(h) further presents computation time t_{comp} and uncertainty estimation error under different sample numbers M . t_{comp} refers to the actual time of running a simulation step. For each case, five tests are performed. The mean and standard deviations of the five computation times are shown in solid red line and red shade, respectively. $\hat{\Delta}_{\text{MAE}}$ denotes the MAE of uncertainty estimation error $\hat{\Delta}$ of 10 s after the learned model is employed. It is obvious that with M increasing, the computation time gets longer, but the estimation accuracy is higher. The computation time of the proposed EVOLVER can fully meet the requirements for online control.

B. Trajectory Prediction of an Irregular Object

EVOLVER is applied to predict the future trajectory of an *uneven* free-flying object, to demonstrate its performance when encountering the uncertainty of $\dot{\Delta} = h(\Delta, x)$.

1) *Problem Formulation*: The dynamics of an *irregular* object is formalized as

$$\begin{cases} \dot{p} = v, \dot{v} = a \\ a = g + \Delta \end{cases} \quad (27)$$

with position $p \in \mathbb{R}^3$, velocity $v \in \mathbb{R}^3$, acceleration $a \in \mathbb{R}^3$, acceleration of gravity $g = [0, 0, g]^T \in \mathbb{R}^3$, and uncertainty $\Delta \in \mathbb{R}^3$. The uncertainty $\Delta \in \mathbb{R}^3$ results from the sophisticated aerodynamic drag due to the *irregular* shape. We use the rotation matrix $R \in \mathbb{R}^{3 \times 3}$ to denote the attitude of the object. The drag Δ of the *irregular* object depends on the velocity and attitude of the object [34], [35]. Hence, the uncertainty in this case is modeled as the type of $\dot{\Delta} = h(\Delta, x)$, where x contains the translational and rotational states of the object.

If the current acceleration a_0 is known, the position after Δt can be predicted according to current states, i.e.,

$$p(t_0 + \Delta t) = p_0 + v_0 \Delta t + \frac{1}{2} a_0 \Delta t^2 \quad (28)$$

where t_0 , p_0 , and v_0 denote the current time, position, and velocity, respectively. A more rigorous prediction strategy is provided in Section VI-B4. The position p_0 and velocity v_0 can be accurately measured by the motion capture system. The objective of the prediction study is to accurately obtain current a_0 by *model-based* or *data-driven-based* methods. Our EVOLVER provides an online learning way to the estimation of Δ and the improvement of the current a_0 estimate.

2) *Implementation Details*: An *irregular* empty bottle, as shown in Fig. 1(a), is selected in this experiment. Several advanced methods are taken as comparisons, including drag coefficient fitting method [59], [60], [61] and machine learning-based method [62], [63]. In total, 30 real free-flying trajectories are recorded by the motion capture system, 21 of these trajectories are taken as the training sets for offline comparison methods excluding the proposed one and the rest of the trajectories are used as the testing set for all of the chosen methods. The real acceleration a in the offline training set is obtained by the LS polynomial fit. Technical details for all implemented methods, including the proposed one, are listed as follows.

- 1) EVOLVER: Δ is estimated by Algorithm 1 as the object is thrown out. The lifting functions $\Phi(\Delta)$ are chosen as $[\Delta_1, \Delta_2, \Delta_3]^T$, whereas the lifting functions $\Psi(\Delta, z)$ are set as same as the input variables of followed machine learning-based method, to make a fair comparison. Due to the short duration (around 1 s) of the whole mission, the uncertainty begins to be learned before enough data has been collected. After the learning dataset $\mathcal{D}_2$ has collected more than $\frac{M}{2}$ samples (by setting $T_1 = \frac{M}{2} t_s$), Δ begins to be learned. Once the learning dataset $\mathcal{D}_2$ has collected more than M samples, Algorithm 2 is enabled to construct the richness dataset. The related parameters are summarized in Table I.

- 2) Drag model-based method [59], [60], [61]: Δ is modeled as the quadratic drag, which is proportional to the square of v , i.e., $a = g - D \|v\| v$, where $D \in \mathbb{R}^{3 \times 3}$ is a symmetric positive matrix, denoting the drag coefficient. D is fitted by the LS algorithm with the training set, which is shown in Table I.
- 3) Machine learning-based method [62], [63]: The dynamics of the object is modeled as $a = f(p, v, R)$ where the unknown nonlinear mapping $f(\cdot)$ is learned by support vector regression (SVR) [64]. To improve learning efficiency, different from [62] and [63], the variable set $\{p, v, v^2, R\}$ is taken as training input, where the square of v is considered additionally. The linear kernel is adopted here.
- 4) Others: A version only considering the gravity and a version enhanced with the normal disturbance observer [20] are conducted as the benchmarked comparisons.
- 3) *Experimental Results*: Fig. 6 shows the experimental results. Nine testing trajectories collected by the motion capture system are presented in Fig. 6(a). For the convenience of data processing, the flight times for all trajectories are tailored to 1.057 s by setting start points. The prediction horizon is set as 0.5 s. The prediction errors are illustrated in Fig. 6(b), where the colored shaded area represents the standard deviations of the testing trajectories. It can be obtained that our EVOLVER method, as well as the drag model-based method and SVR-based method can achieve decent performance compared with the two other benchmarked methods (gravity-base and normal disturbance observer-based methods). The SVR-based method shows the most stable performance. With respect to EVOLVER, since the uncertainty model has not been learned before 0.05 s, its performance mainly depends on the initial uncertainty observer (6). After the learned uncertainty model is employed, the prediction error can converge to a level comparable to offline learning methods. Especially, the online learning regime in EVOLVER can quickly adapt to the changing environment, leading to slightly better performance than offline learning methods toward the end.

Furthermore, the efficiency of the designed dataset-construction method in Algorithm 2 is verified. The trajectory darkened in Fig. 6(a) is selected to illustrate the result. We implement EVOLVER to predict the object trajectory with and without Algorithm 2, respectively. Fig. 6(c) shows the benefits of our designed dataset-construction method. The constructed dataset in the final prediction step is labeled on the trajectory. When the information index $\mathfrak{S}$ is not considered, the samples in the training dataset mainly concentrate around the current state, resulting in ill-fitting and failed estimation convergence as highlighted in Fig. 6(d). After the information index $\mathfrak{S}$ is involved, the samples cover richer states, which becomes sparse with the *timeliness* decreasing.

- 4) *Further Improvement*: Experimental results show that there is still an error of 0.1 m among the drag model-based, SVR-based, and EVOLVER methods. The reason can be attributed to simple Euler integration used in (28) at each prediction step, which assumes that the resultant force of the followed trajectory is the same as the one at the current prediction moment. The

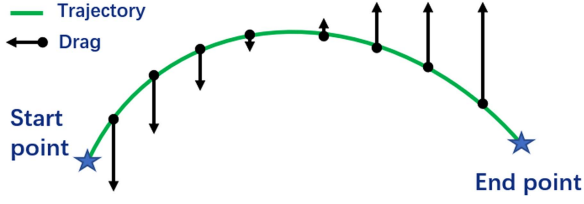


Fig. 7. Conceptual graph: Coarse variation of the aerodynamic drag along the height direction. The direction of the drag is opposite to the object velocity. The faster the speed, the greater the magnitude of the drag.

differential (27) does not have a closed-form solution in general as a result of the complex structure of Δ . Rigorous step-by-step forward integration would induce high computation cost [65]. To save the computational cost, the Euler integration (28) is used in most previous works [59], [61], [63], [66], [67]. In [65], a closed-form approximation of the flight trajectory is derived for a 2-D moving regular object. Here we provide another prediction strategy for the *irregular* object by employing the inherent property of the proposed EVOLVER.

Fig. 7 illustrates the coarse variety of the aerodynamic drag along the height direction. The vertical motion is approximately a motion with a constant jerk. To examine this intuitive observation, polynomials of different degrees are used to fit all recorded collected trajectories. The fitting results are presented in Fig. 6(e). The third-order fitting (i.e., acceleration varies uniformly) has a satisfactory performance with the lower order. Thus, the assumption of constant jerk motion is more reasonable than the assumption of constant acceleration motion in (28). Fortunately, the proposed EVOLVER provides $\dot{\Delta}$ estimation in (17), i.e., the jerk of *irregular* object system (27). Denoting j_0 as the current jerk, (28) is adjusted to

$$\mathbf{p}(t_0 + \Delta t) = \mathbf{p}_0 + \mathbf{v}_0 \Delta t + \frac{1}{2} \mathbf{a}_0 \Delta t^2 + \frac{1}{6} \mathbf{j}_0 \Delta t^3. \quad (29)$$

Fig. 6(f) shows the result that the prediction mechanism is switched from (28) to (29) at $t = 0.25$ s. The prediction error is found to have further decreased after involving the jerk information.

C. Application to a Quadrotor

In this experiment, EVOLVER is applied for an aggressively flying quadrotor under the wind disturbance, which belongs to the type of $\dot{\Delta} = \mathbf{h}(\Delta, \mathbf{x}, \mathbf{l}_u)$. By employing the proposed EVOLVER, the quadrotor can well adapt to external wind disturbance, in comparison with several methods [20], [36].

1) *Problem Formulation*: The kinematics and dynamics of the translational loop are formalized as

$$\begin{cases} \dot{\mathbf{p}} = \mathbf{v}, \dot{\mathbf{v}} = \mathbf{a} \\ m\mathbf{a} = -f\mathbf{b}_3 + mg\mathbf{e}_3 + \Delta \end{cases} \quad (30)$$

with position $\mathbf{p} \in \mathbb{R}^3$, velocity $\mathbf{v} \in \mathbb{R}^3$ [i.e., $\mathbf{x}$ in (1)], acceleration $\mathbf{a} \in \mathbb{R}^3$, acceleration of gravity $\mathbf{g} \in \mathbb{R}^3$, mass m , the total thrust magnitude $f \in \mathbb{R}$, inertial frame $\mathcal{I} = [\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3]$, body frame $\mathcal{B} = [\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3]$, and the wind disturbance $\Delta \in \mathbb{R}^3$. The frame definition is further illustrated in Fig. 8(a).

Different from the last object trajectory prediction example where $\dot{\Delta} = \mathbf{h}(\Delta, \mathbf{x})$, in the quadrotor case, Δ additionally depends on external wind speed, which is unknown for the quadrotor [34], [35], [36]. Thus, the uncertainty is considered as the type of $\dot{\Delta} = \mathbf{h}(\Delta, \mathbf{x}, \mathbf{l}_u)$ where $\mathbf{l}_u$ represents the relevant part of unknown wind speed.

2) *Implementation Details*: The control framework of the quadrotor is shown in Fig. 8(b). The controller in our previous work [36] in the absence of the drag utilization and the wind speed observer (WSO) parts is employed as the baseline controller.

In indoor experiments, the quadrotor is commanded to follow a circle trajectory $[r \sin(\frac{2\pi}{T}t) \ r \cos(\frac{2\pi}{T}t) \ 1]^\top \text{ m}$, where the radius r and the period T are set as 1 m and 2.1 s, respectively, i.e., 3 m/s speed and 9 m/s² acceleration. Two 380 W fans are placed at $[-2.5, 0.5, 1] \text{ m}$ and $[-2.5, -0.5, 1] \text{ m}$, along the direction of $\mathbf{e}_1$. Fig. 8(e) illustrates the indoor wind field distribution, which is obtained by spatial sampling through a digital anemometer AS8556. Notice that the sampling altitude is consistent with that of the quadrotor flight. The maximum wind speed is up to 6 m/s. In particular, a foam board (0.125 m $\times$ 0.465 m) is attached to the back of the quadrotor (facing to the fans) to increase the drag surface. The wind disturbance is mainly in the direction of $\mathbf{e}_1$. Thus, the uncertainty along $\mathbf{e}_1$, denoted as Δ_x , is studied. In outdoor experiments, the same flight mission is conducted within a natural wind field. The maximum wind speed is up to 5 m/s.

Apart from the EVOLVER, several methods (mainly focus on online paradigm) are implemented as comparisons: the baseline controller, the normal disturbance observer-based method [20], and the WSO-based method [36]. Technical details for all compared methods are listed as follows.

- 1) EVOLVER: In the indoor experiment, the wind field produced by two fans is approximately constant. Thus, the influence caused by $\mathbf{l}_u$ is disregarded. The lifting function $\mathbf{S}$ is chosen as $[\Delta_x, v_x, \theta, \|v_x\|v_x, \Delta_x\theta, qv_x]^\top$, where v_x , θ , and q denote the velocity along $\mathbf{e}_1$, the pitch, and the attitude velocity along $\mathbf{b}_2$ of the quadrotor, respectively. In the outdoor experiment, considering the time-varying property of wind speed, according to Section IV-E more historical uncertainty states (n_Δ is set as 2) are added in the lifting function to attenuate the influence of $\mathbf{l}_u$. Algorithm 1 is implemented as the quadrotor begins its flight trajectory. The related parameters of the EVOLVER are summarized in Table I. Since the diversity of the dataset can be guaranteed by setting T_1 greater than the trajectory period, we disable the dataset construction process in this example.
- 2) The baseline controller: The controller presented in Fig. 8(b) without the uncertainty observer is implemented. In the absence of the wind disturbance, the tracking error can reach 6 cm (RMSE) at 4 m/s and 13.23 m/s².
- 3) The normal disturbance observer-based method [20]: With the assumption of $\dot{\Delta} = 0$, the observer (19) is implemented to estimate the wind disturbance.
- 4) The WSO-based method [36]: The wind disturbance is modeled as $\mathbf{F}_w = -\mathbf{RDR}^\top \mathbf{v}_a = -\mathbf{RDR}^\top \mathbf{v} +$

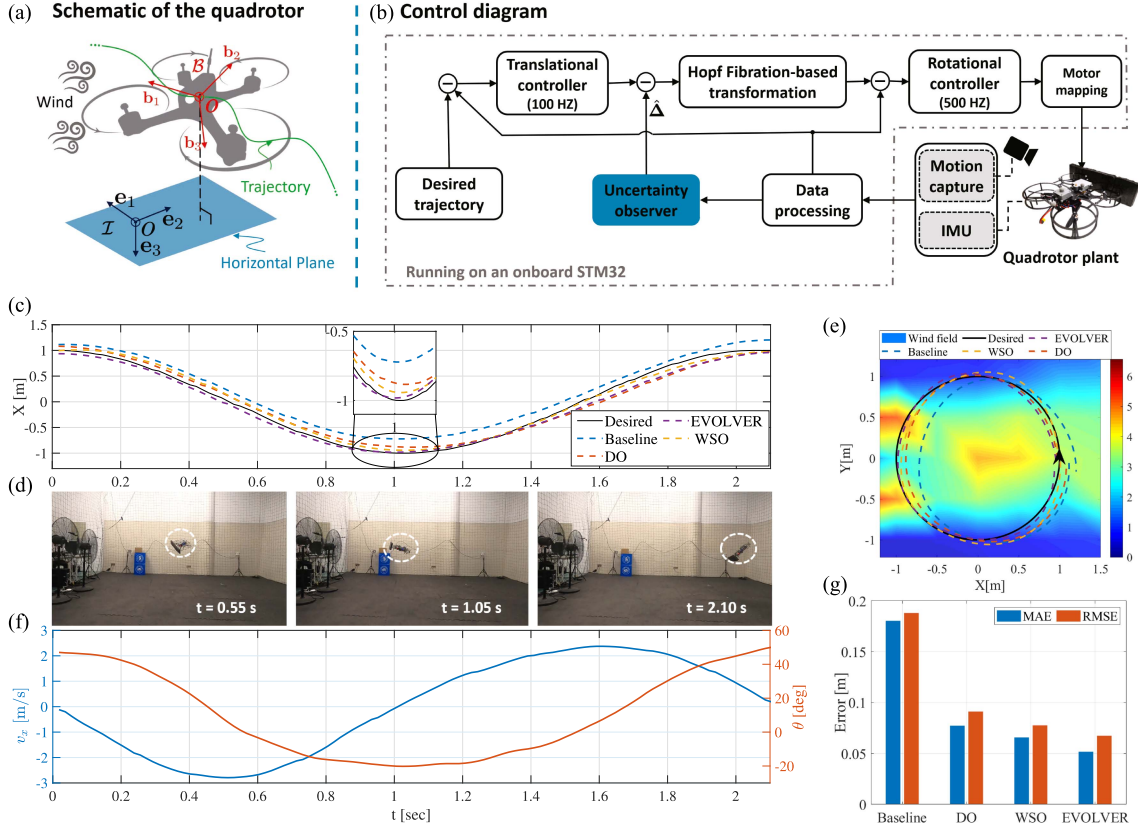


Fig. 8. Experimental results in the quadrotor example. (a) Schematic of the employed 6-inch quadrotor. (b) Control diagram. The baseline controller in our previous work [36] is employed, which consists of a PD translational controller with an acceleration feedforward, a Hopf fibration-based transformation [68], and a geometric rotational controller. The wind disturbance is estimated by the designed uncertainty observer and is subsequently compensated in the baseline controller. All control algorithms are implemented on an onboard low-computing-power processor. (c) Tracking performance in the direction of e_1 of all compared methods: the proposed EVOLVER, the baseline controller, the normal disturbance observer-based method [20], and the WSO-based method [36], abbreviated to EVOLVER, baseline, DO, and WSO, respectively. (d) Three snapshots of the agile flight. (e) 2-D tracking performance of all compared methods in the wind field. (f) Evolution of measured velocity and attitude along e_1 . The maximum attitude can reach 50° . (g) Quantitatively compared result of all implemented methods over four laps.

$RDR^T v_w \in \mathbb{R}^3$ [34], where $D \in \mathbb{R}^{3 \times 3}$ represents drag coefficients, $R \in \mathbb{R}^{3 \times 3}$ denotes rotation matrix from body frame to inertial frame, v_a denotes the relative airspeed, and v_w denotes the wind speed. In [36] a drag utilization method is employed to utilize the part $-RDR^T v$ and a WSO is designed to compensate the part $RDR^T v_w$. The knowledge of $-RDR^T$ is utilized in the WSO, reducing the conservativeness of the normal disturbance observer-based method. By fitting prior data, the drag coefficient D can be obtained, which is shown in Table I.

To make a fair comparison, the observer gains of all compared methods are set as the same. The positions of the quadrotor are measured by a motion capturing system (indoor) and a Real Time Kinematic positioning system (outdoor), which are transmitted to the quadrotor through an ultrawideband transmission module at 50 Hz. The angular rate is measured by onboard IMU at 500 Hz. The whole control algorithm shown in Fig. 2 is implemented on an STM32F7 processor.

3) *Experimental Results:* Fig. 8(c)–(g) shows the indoor experimental results of the quadrotor. The trajectory tracking performances in the steady state of all implemented methods are presented in Fig. 8(c) and Fig. 8(e). The transient performance of

the EVOLVER is omitted, which is similar to the normal disturbance observer-based one. It can be seen that the trajectory shifts a lot along the wind direction under the baseline controller. As the normal disturbance observer is enabled, the wind disturbance is partially attenuated. However, since the normal disturbance observer is only suitable for the slow variable disturbance, the wind disturbance cannot be accurately estimated when the attitude of the quadrotor changes quickly. After the knowledge of $-RDR^T$ is utilized, i.e., the WSO is activated, the trajectory tracking performance is improved. However, two drawbacks exist when employing the WSO. The first is the drag coefficient D needs to be obtained beforehand. The other is that the first-order disturbance model F_w is simplified. There may be other factors that affect the wind disturbance, such as the velocity and the attitude velocity of the quadrotor. The EVOLVER addresses the aforementioned issues. The flight trajectories and the preset circle path overlap more precisely as EVOLVER works. The quantitatively compared result is presented in Fig. 8(g), which evaluates MAE and RMSE of all methods over four laps.

Three snapshots of the agile flight are presented in Fig. 8(d). Moreover, Fig. 8(f) shows the evolution of the pitch θ and the velocity v_x of the quadrotor. In the flight test, the maximum pitch can reach 50° . Fig. 9 shows the outdoor experimental results.

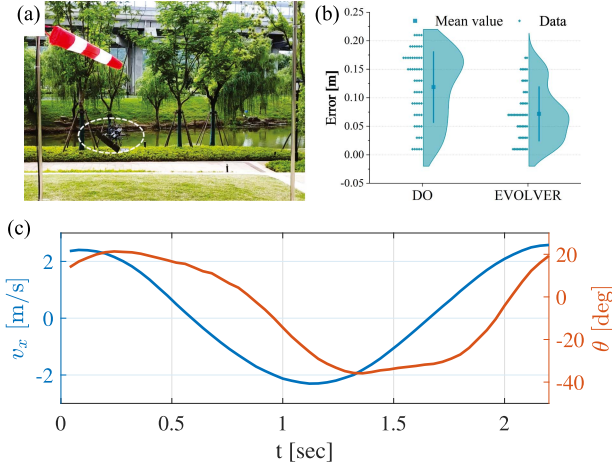


Fig. 9. Outdoor experimental result of the quadrotor example. (a) Outdoor experimental scenario. (b) Quantitative comparison of implemented methods (the proposed EVOLVER and the normal disturbance observer-based method [20], abbreviated to EVOLVER and DO, respectively). (c) Evolution of measured velocity and attitude along e_1 .

The violin plot in Fig. 9(b) further demonstrates the effectiveness of the EVOLVER, compared with the normal disturbance observer-based method [20], with 39.5% improvement in MAE. The evolution of the pitch θ and the velocity v_x are depicted in Fig. 9(c).

D. Application to a Manipulator

Finally, the end-effector trajectory tracking of the manipulator subject to unwanted base motion is considered. In this mission, the proposed EVOLVER is employed to estimate the uncertainty induced by the base motion, which belongs to the type of $\hat{\Delta} = h(\Delta, x, \ell_k, \ell_u)$.

1) *Problem Formulation:* As shown in Fig. 10(a), the position of the end-effector $p_{\mathcal{I}}^e \in \mathbb{R}^3$ expressed in the inertial frame $\mathcal{I}$ can be formalized as

$$p_{\mathcal{I}}^e = p_{\mathcal{I}}^b + R p_{\mathcal{B}}^e \quad (31)$$

where $p_{\mathcal{I}}^b \in \mathbb{R}^3$ denotes the position of the base expressed in $\mathcal{I}$, $p_{\mathcal{B}}^e \in \mathbb{R}^3$ represents the position of the end-effector expressed in the base frame $\mathcal{B}$, and $R \in \mathbb{R}^{3 \times 3}$ is the rotation matrix from body/base frame to inertial frame.

The objective is to make sure that the position of the end-effector $p_{\mathcal{I}}^e$ tracks a given desired trajectory $p_{\mathcal{I}}^{e,d} \in \mathbb{R}^3$ by controlling the joint velocity $\dot{q} \in \mathbb{R}^{n_q}$ for a n_q -DOF manipulator. Thus, the kinematics of the manipulator is needed. By differentiating (31), it can be obtained that

$$v_{\mathcal{I}}^e = R v_{\mathcal{B}}^e + v_{\mathcal{I}}^b + \omega_{\mathcal{B}} \times R p_{\mathcal{B}}^e \quad (32)$$

where $v_{\mathcal{I}}^e \in \mathbb{R}^3$ denotes the velocity of the end-effector expressed in $\mathcal{I}$, $v_{\mathcal{B}}^e \in \mathbb{R}^3$ is the velocity of the end-effector expressed in $\mathcal{B}$, $v_{\mathcal{I}}^b \in \mathbb{R}^3$ represents the velocity of the base expressed in $\mathcal{I}$, and $\omega_{\mathcal{B}} \in \mathbb{R}^3$ is the attitude velocity of the base expressed in $\mathcal{B}$.

Similar to [54], we assume the position and attitude of the base can be measured, whereas their derivatives cannot be obtained

accurately. In other words, $v_{\mathcal{I}}^b + \omega_{\mathcal{B}} \times R p_{\mathcal{B}}^e$ in (32) is unknown. Here, $v_{\mathcal{I}}^b + \omega_{\mathcal{B}} \times R p_{\mathcal{B}}^e$ is treated as the uncertainty Δ , which belongs to the type of $\hat{\Delta} = h(\Delta, x, \ell_k, \ell_u)$. The EVOLVER is employed to learn it with $x = \{p_{\mathcal{B}}^e\}$, $\ell_k = \{R\}$, and $\ell_u = \{v_{\mathcal{I}}^b, \omega_{\mathcal{B}}, \dots\}$ in (2). Then, (32) can be rewritten as

$$v_{\mathcal{I}}^e = R v_{\mathcal{B}}^e + \Delta. \quad (33)$$

2) *Controller Design:* The overall control framework for the manipulator is illustrated in Fig. 10(b). Define the tracking error of the end-effector as $e_p = p_{\mathcal{I}}^{e,d} - p_{\mathcal{I}}^e$. By recalling (33), the proportional–integral (PI) controller is designed as

$$v_{\mathcal{B}}^{e,d} = R^T \left(v_{\mathcal{I}}^{e,d} + K_p e_p + K_i \int e_p dt - \hat{\Delta} \right) \quad (34)$$

where $v_{\mathcal{B}}^{e,d} \in \mathbb{R}^3$ and $v_{\mathcal{I}}^{e,d} \in \mathbb{R}^3$ denote the desired velocities of the end-effector expressed in $\mathcal{B}$ and $\mathcal{I}$, respectively. $K_p \in \mathbb{R}^{3 \times 3}$ and $K_i \in \mathbb{R}^{3 \times 3}$ are the control gains of the PI controller.

In consequence, the desired joint velocity $\dot{q}^d \in \mathbb{R}^{n_q}$ is rendered as

$$\dot{q}^d = J^\dagger \left[v_{\mathcal{B}}^{e,dT}, \omega^{e,dT} \right]^T \quad (35)$$

where $J \in \mathbb{R}^{n_q \times n_q}$ is the Jacobian matrix, and $\omega^{e,d} \in \mathbb{R}^3$ denotes the desired attitude velocity of the end-effector.

In experiment, the control input $\omega^{e,d}$ of the attitude loop in (35) is also designed as the PI controller [54]. The attitude antidisturbance problem of the end-effector is not specifically considered to simplify the process. Our proposed EVOLVER can be easily transformed to the attitude loop by referring to the design process of the position loop.

3) *Implementation Details:* The UR-CB3 manipulator (see Fig. 1) with 6-DOF (i.e., $n_d = 6$) is employed in this test. The end-effector is commanded to be fixed at a certain position with respect to the inertial frame $\mathcal{I}$. The joint velocity control model is employed. The states of the base and the end-effector are measured by the motion capture system and the inner sensors of the manipulator, respectively. All the involved schemes are implemented in Python running on a PC with Inter(R) Core(TM) i7-8700 CPU.

Several comparison methods are also implemented to demonstrate the advantages of the proposed EVOLVER: the PI controller, the normal disturbance observer-based method [20], and the predictive end-effector control method [54]. Technical details for the chosen methods are listed as follows.

- 1) **EVOLVER:** According to the property of Δ in (33), the lifting functions S are chosen as $[\Delta^T, \Delta_{T_1}^T, \dots, \Delta_{T_{n_d}}^T, p_{\mathcal{I}}^b]^T$. Algorithm 1 is implemented as the base begins to move. The related parameters of the EVOLVER are listed in Table I. According to the quasiperiodic property of the autonomous underwater vehicle (AUV) motion [see Fig. 10(c)], the diversity of the dataset can be guaranteed by setting T_1 greater than the approximate period. Thus, the dataset construction process in this example is disabled in real test. The dataset construction process is invoked when evaluating the computational cost.
- 2) **The PI Controller:** The controller (34) without considering uncertainty compensation (i.e., $\hat{\Delta} = 0$) is implemented.

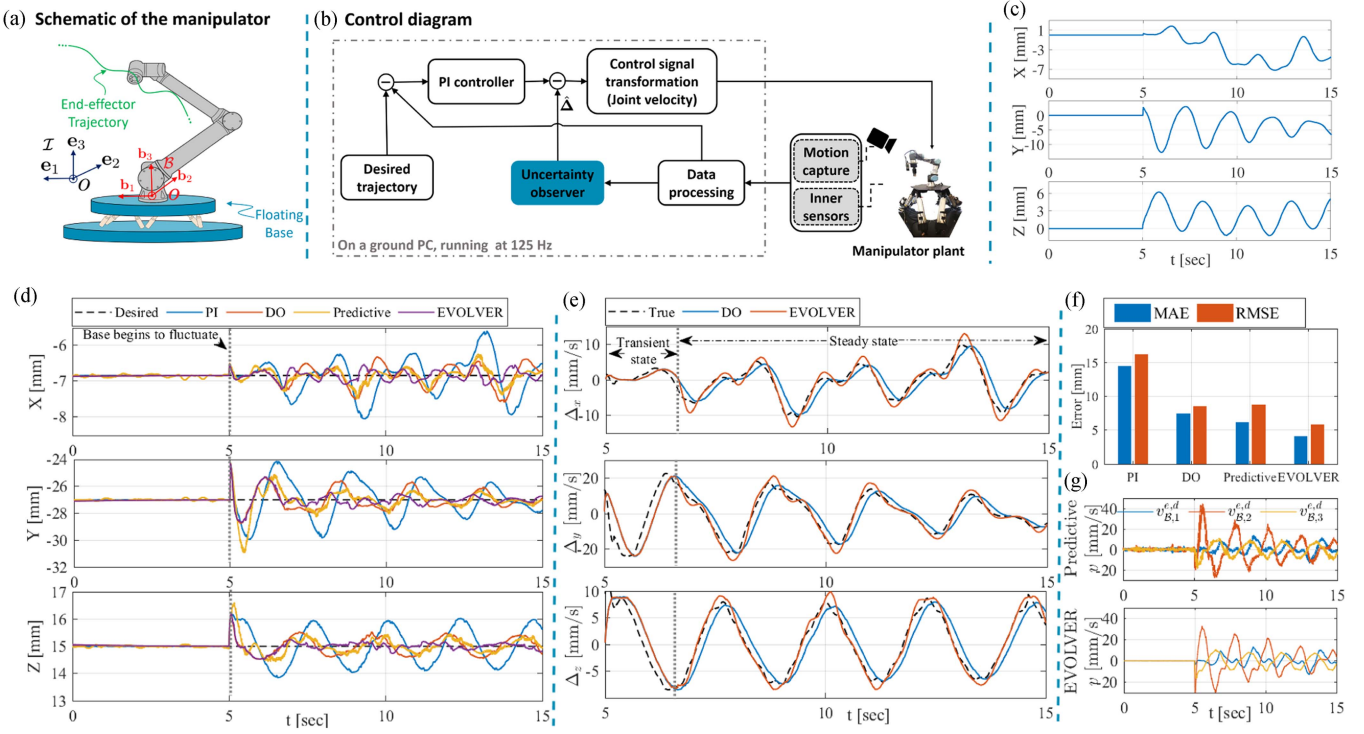


Fig. 10. Experimental results of *Scenario #1* in the manipulator example. (a) Schematic of the employed UR-CB3 manipulator. (b) Control diagram. The PI baseline controller is employed. The Position and attitude of the base are measured by the external motion capture system, and the position and attitude of the end-effector are obtained from the inner sensors. The desired joint velocity from the controller is sent to the manipulator finally. (c) Translational motion of a real AUV while under wave excitation in shallow water, which is extracted from Fig. 2 of [54]. (d) Tracking performance of the end-effector under different schemes: the proposed EVOLVER, the PI controller, the normal disturbance observer-based method [20], and the predictive control method [54], abbreviated to EVOLVER, PI, DO, and Predictive, respectively. (e) Estimation uncertainties of the normal disturbance observer-based method and the proposed EVOLVER. (f) Quantitatively compared result of all implemented methods. (g) Controller outputs $\mathbf{v}_B^{e,d} = [v_{B,1}^{e,d}, v_{B,2}^{e,d}, v_{B,3}^{e,d}]^T$ of the predictive control method and the proposed EVOLVER.

- 3) The Normal Disturbance Observer-Based Method [20]: With the assumption of $\dot{\Delta} = 0$, the observer (19) is implemented to estimate uncertainty induced by the base movement. Then, the estimated uncertainty is compensated in the PI baseline controller (34). The observer gain is set as same as the EVOLVER.
- 4) The Predictive Control Method [54]: The base movement is forecasted by linearly fitting the historical data, and the desired state of the end-effector with respect to the base frame is predicted. The end-effector tracking problem is finally formalized as a MPC framework. Consistent with our control scheme, the control signal to the manipulator is the joint velocity. The controller parameters are set as same as the unconstrained situation from [54]. For the parameters unmentioned in [54], the number of historical base data selected for fitting is set as $n_{\Delta} = 5$ in keeping with the EVOLVER, and the forecast horizon is set as 10.

Two scenarios are arranged to carry out the tests.

Scenario #1: In pursuit of authenticity, the base disturbance simulates a real AUV motion while under wave excitation in shallow water, as shown in Fig. 10(c), which is collected from [54]. The base disturbance is introduced at $t = 5$ s. In pursuit of comparison fairness, the base motion needs to be accurately reproduced for all test methods. Whereas, the employed 6-DOF floating base can only be controlled by a rocker. In this scenario, we impose the base disturbance in a virtual way.

Imagine the base positioning system is moving, i.e., the base is fixed with respect to the ground station and the inertial frame is commanded to follow the opposite trajectory of Fig. 10(c). In such a way, the position of the end-effector must follow the opposite trajectory of Fig. 10(c), i.e., at rest with the moving inertial system.

Scenario #2: A more challenging condition is considered to examine the applicability of the proposed scheme. The manipulator is commanded to operate on a parallel 6-DOF floating base. A 6-DOF rocker is used to control the translational and rotational movements of the base, as shown in Fig. 11(a). In the test, the EVOLVER method and the predictive control method [54] are implemented. The parallel 6-DOF floating base in our lab is controlled by the 6-DOF rocker manually. The perturbations are not guaranteed to be 100% reproducible. In the experiment, the perturbations with similar magnitude and frequency are tried to be applied for different methods. The attitude prediction strategy proposed in [54] is considered in the predictive control method. For the EVOLVER, the lifting functions are selected as in *Scenario 1* (i.e., without considering the attitude information of the floating base), to illustrate the robustness of the proposed framework.

4) *Experimental Results*: The experimental results of the two scenarios are shown in Figs. 10 and 11, respectively.

Scenario #1: Fig. 10(d) presents the tracking performance of the end-effector under different control schemes and Fig. 10(f)

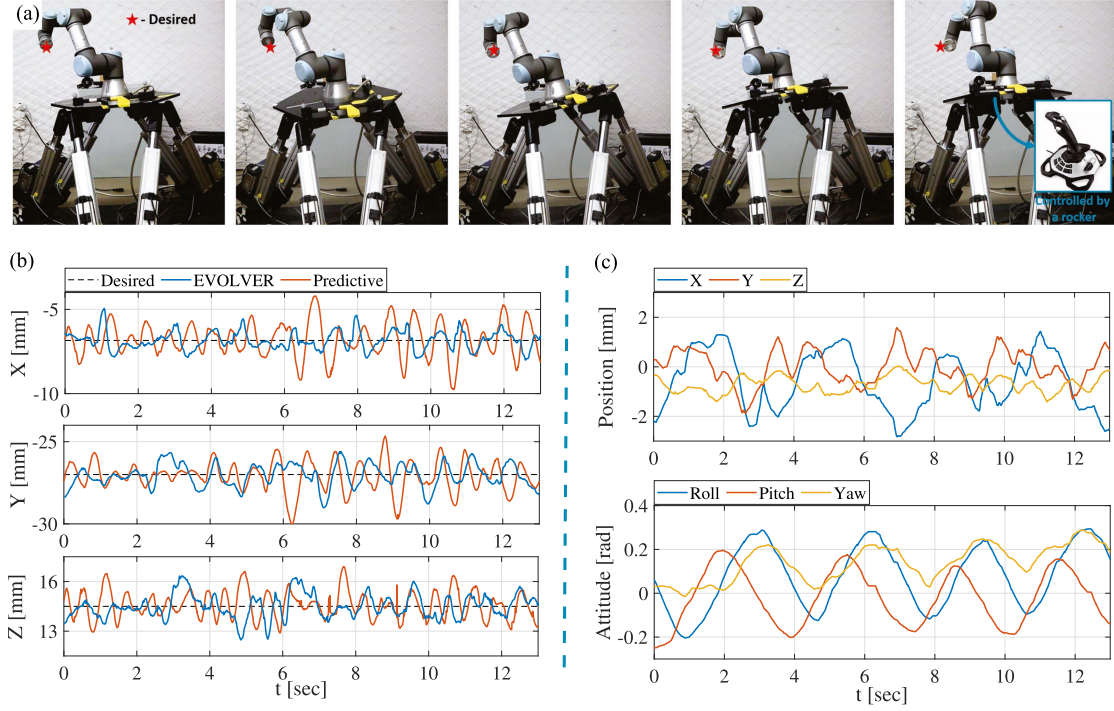


Fig. 11. Experimental results of *Scenario #2* in the manipulator example. (a) Five snapshots of the operating manipulator on a parallel 6-DOF floating base controller by a 6-DOF rocker. (b) Steady-state tracking performance of the end-effector under different schemes: the proposed EVOLVER and the predictive control method [54], abbreviated to EVOLVER and Predictive, respectively. (c) Real 6-DOF moving trajectory of the base.

shows the quantitative comparison results. In the presence of the same uncertainty, our proposed EVOLVER excels the PI control method, the normal disturbance observer-based method, and the predictive control method, with 71.8%, 45.8%, and 33.9% improvement in MAE, respectively.

The estimated uncertainties by the normal disturbance observer-based method and the EVOLVER are depicted in Fig. 10(e). Notice that the truth value of the uncertainty is fitted offline using the polynomial fitting method. At the transient stage, the estimation performances of both methods are the same with a certain delay. After the learned uncertainty model is employed at $t = 6.6$ s, the estimation delay is eliminated immediately, which confirms that the EVOLVER has successfully predicted the motion of the base. Moreover, the controller outputs of the predictive control method and the EVOLVER are provided in Fig. 10(g). In comparison with the predictive control method, the EVOLVER can output smoother control signals to achieve better performance. The output of the predictive control is chattering, which is harmful to the actuators. Part of the reason can be attributed to the discontinuity of the predicted uncertainty, where the coefficients are discontinuously updated by linear LS.

Scenario #2: Figs. 1(c) and 11(a) present the long-exposure photo and five snapshots of the manipulator response with the EVOLVER method, respectively. Even though the base is moving randomly with 6-DOF [the real moving trajectory of the base is provided in Fig. 11(c)], the end-effector can still maintain at the desired position. The steady-state performance is shown in Fig. 11(b). The MAE and RMSE of the predictive control method

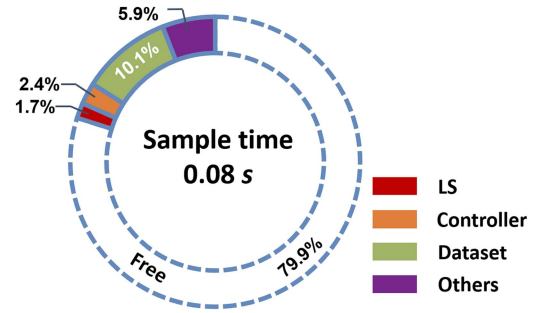


Fig. 12. Computational cost ratio of various modules. Controller: The PI baseline controller (34) and the observer (19). Dataset: the RD (22) and dataset construction flow stated in Algorithm 2. LS: the LS solving step (11) and the continuous step (14). Others: signal filtering and so on.

are 0.013 m and 0.015 m, respectively. By contrast, the MAE and RMSE of the EVOLVER are 0.009 m and 0.010 m, which improve 30.8% and 33.3% as compared with the predictive control method, respectively.

Finally, Fig. 12 exhibits the computational cost ratio of various modules. Especially, the dataset construction process designed in Section V-A has an acceptable computational cost of 10.1%, which is suitable for the online situation.

VII. CONCLUSION

In this article, we have made significant steps toward on-line learning and prediction of disturbances for various robotic

systems. We achieve this by using the Koopman operator to model the unknown uncertainty and substituting the learned model into an evolutionary uncertainty observer. The presented scheme enables rapid transient reaction ability and high-precision steady-state tracking performance. The convergence guarantee can be preserved in several optimal conditions. To achieve these optimal requirements in real applications, practical solutions including training dataset construction, data noise handling, and lifting functions selection are provided. Finally, the proposed framework is demonstrated by sufficient simulation and experiments: stable control of a second-order *Newton* system with the chaotic *Lorenz* uncertainty, trajectory prediction of an irregular free-flying object under the aerodynamic drag, agile flight of the quadrotor under the wind disturbance, and end-effector control of the manipulator under unwanted base moving disturbance.

APPENDIX

A. Reasonability and Limitation of Assumption 1

The reasonability and limitation of Assumption 1 are analyzed by combining the studied examples.

Reasonability: In the Lorentz example, from (25), it can be seen that the injected Lorentz uncertainty satisfies Assumption 1. In the irregular object and quadrotor examples, the uncertainties mainly result from the air drag, which can be roughly modeled as $F_w = -RDR^T v_a = -RDR^T v + RDR^T v_w \in \mathbb{R}^3$ [34], [35], [36], where $D \in \mathbb{R}^{3 \times 3}$ represents drag coefficients, $R \in \mathbb{R}^{3 \times 3}$ denotes rotation matrix from body frame to inertial frame, v_a denotes the relative airspeed, v denotes the velocity, and v_w denotes the wind speed. In the irregular object example ($v_w = 0$), the system states of the object are bounded in reality. Assumption 1 is always held in this case.

In the quadrotor example ($v_w \neq 0$), the air drag depends on the translational and rotational states, and the external wind speed. The boundness of the rotational states can be guaranteed by a stable rotational controller (e.g., the geometric controller [23]). For the translational state, it can be initially bounded by a baseline translational controller (e.g., the PD controller) despite of a large tracking error. After employing EVOLVER, with the decreasing of the uncertainty estimation error at each step, the change of translational state would not break through Assumption 1. Finally, we only need to hold the assumption that the variation of wind speed is bounded, which is acceptable in practice. With respect to the manipulator example, by following a similar analysis, Assumption 1 finally requires the variation of the base floating to be bounded.

The true uncertainties of experimental examples under baseline control/prediction algorithms are provided in Fig. 13(a), (b), and (e), respectively, which can certify the reasonability of Assumption 1 from a practical aspect.

Limitation: Assumption 1 is not applicable to the disturbance like push or collision. The future work will pursue separating the part related to the system state from uncertainty based on *Chebyshev* polynomials [69], and reduce the conservativeness of Assumption 1.

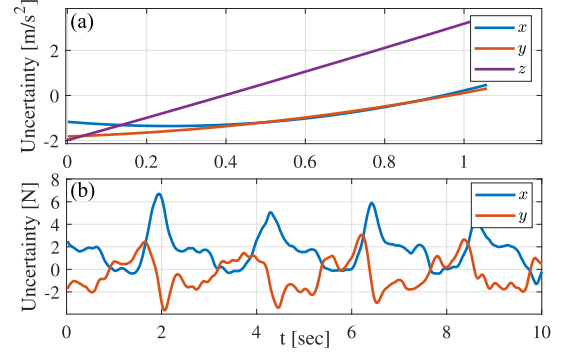


Fig. 13. Truth uncertainties in the object trajectory prediction and quadrotor examples under baseline control/prediction algorithms. Notice that the truth uncertainties are fitted offline by polynomial fitting. (a) Object trajectory prediction example. (b) Quadrotor example.

B. Proof of Lemma 1

Proof: Differentiate $\tilde{\Delta}$. From (6), it can be implied that

$$\begin{aligned} \dot{\tilde{\Delta}} &= \dot{\Delta} - (\dot{\xi} - \dot{p}(x)) \\ &= \dot{\Delta} + L\dot{x} - L(f_x(x) + f_u(x)u + \hat{\Delta}) \\ &\stackrel{(a)}{=} \dot{\Delta} + L\tilde{\Delta} \end{aligned} \quad (36)$$

where (a) is obtained by using (1). Define the *Lyapunov* candidate $V_1 = \frac{1}{2} \tilde{\Delta}^T \tilde{\Delta}$. Differentiating V_1 , one can obtain

$$\begin{aligned} \dot{V}_1 &= \tilde{\Delta}^T L\tilde{\Delta} + \tilde{\Delta}^T \dot{\tilde{\Delta}} \\ &\stackrel{(b)}{\leq} \delta \tilde{\Delta}^T \tilde{\Delta} + \epsilon \|\dot{\tilde{\Delta}}\|^2 \\ &\stackrel{(c)}{\leq} \delta \tilde{\Delta}^T \tilde{\Delta} + \epsilon \tilde{\Delta}^2 \end{aligned} \quad (37)$$

where ϵ is an arbitrarily positive number, $\delta = \lambda_M(L) + \frac{1}{4\epsilon}$, (b) results from the conditions $\tilde{\Delta}^T L\tilde{\Delta} \leq \lambda_M(L) \tilde{\Delta}^T \tilde{\Delta}$ and $\tilde{\Delta}^T \dot{\tilde{\Delta}} \leq \frac{1}{4\epsilon} \tilde{\Delta}^T \tilde{\Delta} + \epsilon \|\dot{\tilde{\Delta}}\|^2$ (*Young Inequality*), and (c) results from Assumption 1. From the definition of V_1 , it can be further rendered that

$$\dot{V}_1 \leq 2\delta V_1 + \epsilon \tilde{\Delta}^2. \quad (38)$$

Solving the first-order ordinary differential inequality, one can obtain that

$$0 \leq V_1 \leq e^{2\delta t} \left[V_1(0) + \frac{\epsilon \tilde{\Delta}^2}{2\delta} \right] - \frac{\epsilon \tilde{\Delta}^2}{2\delta}. \quad (39)$$

Thus, if $\delta < 0$, i.e., $\lambda_M(L) < -\frac{1}{4\epsilon}$, we have

$$0 \leq \|\tilde{\Delta}\|^2 \leq 2e^{-2|\delta|t} \left[V_1(0) + \frac{\epsilon \tilde{\Delta}^2}{2\delta} \right] - \frac{\epsilon \tilde{\Delta}^2}{\delta} \quad (40)$$

resulting in $\lim_{t \rightarrow \infty} \|\tilde{\Delta}\| \leq \sqrt{\frac{\epsilon \tilde{\Delta}^2}{|\delta|}}$. In other words, the estimated error of model uncertainty can converge to the set $\sqrt{\frac{\epsilon \tilde{\Delta}^2}{|\delta|}}$. ■

C. Proof of Lemma 3

Proof: The proof procedure is similar to Lemma 1. Combining (17) and (19), it can be implied that

$$0 \leq \|\tilde{\Delta}\|^2 \leq 2e^{-2|\delta|t} \left[V_1(0) + \frac{\epsilon \|C\mathbf{r}_3\|^2}{2\delta} \right] - \frac{\epsilon \|C\mathbf{r}_3\|^2}{\delta}. \quad (41)$$

If the conditions in Lemma 2 are satisfied and ℓ_u is neglected, $\mathbf{r}_3 = 0$ will be achieved. From (41), it is obvious that the estimated error of model uncertainty can converge to zero as $\delta < 0$, i.e., $\lambda_M(L) < -\frac{1}{4\epsilon}$. Here, as $\epsilon \rightarrow \infty$, the required condition can be relaxed to L being negative definite. ■

D. Proof of Theorem 1

Proof: The whole procedure takes place over several discrete stages. The arbitrarily positive number ϵ is chosen as 1.

i) $t \leq T_1$: From Lemma 1, it has been obtained

$$\|\tilde{\Delta}(t)\|^2 \leq 2e^{-2|\delta|t} \left[V_1(0) + \frac{\tilde{\Delta}^2}{2\delta} \right] - \frac{\tilde{\Delta}^2}{\delta}. \quad (42)$$

ii) $T_1 < t \leq T_1 + T_2$: The learned uncertainty model is updated for the first time (i.e., $\mathbf{A}_1$ in lines 12 of Algorithm 1). Similar to (41), it can be rendered that

$$\begin{aligned} \|\tilde{\Delta}(t)\|^2 &\leq 2e^{-2|\delta|t} \left[V_1(T_1) + \frac{\|C\mathbf{r}_3\|^2}{2\delta} \right] - \frac{\|C\mathbf{r}_3\|^2}{\delta} \\ &= 2e^{-2|\delta|t} \left[\frac{1}{2} \|\tilde{\Delta}(T_1)\|^2 + \frac{\|C\mathbf{r}_3\|^2}{2\delta} \right] - \frac{\|C\mathbf{r}_3\|^2}{\delta}. \end{aligned} \quad (43)$$

iii) $T_1 + T_2 < t \leq T_1 + 2T_2$: The learned uncertainty model is updated for the second time after a cycle T_2 . We have

$$\begin{aligned} \|\tilde{\Delta}(t)\|^2 &\leq 2e^{-2|\delta|t} \left[\frac{1}{2} \|\tilde{\Delta}(T_1 + T_2)\|^2 + \frac{\|C\mathbf{r}_3\|^2}{2\delta} \right] - \frac{\|C\mathbf{r}_3\|^2}{\delta}. \end{aligned} \quad (44)$$

iv) $T_1 + jT_2 < t \leq T_1 + (j+1)T_2$: The learned uncertainty model is updated for the j th time (i.e., $\mathbf{A}_j$ in lines 16 of Algorithm 1). We have

$$\begin{aligned} \|\tilde{\Delta}(t)\|^2 &\leq 2e^{-2|\delta|t} \left[\frac{1}{2} \|\tilde{\Delta}(T_1 + jT_2)\|^2 + \frac{\|C\mathbf{r}_3\|^2}{2\delta} \right] - \frac{\|C\mathbf{r}_3\|^2}{\delta}. \end{aligned} \quad (45)$$

The above procedure obeys two principles. One is that the initial estimation error in each period is the final estimation error of the last period. The other one is that the estimation error is asymptotically convergent during each period. Although the updates of the learned uncertainty model are discrete, the estimation error $\tilde{\Delta}$ is continuous and asymptotically convergent.

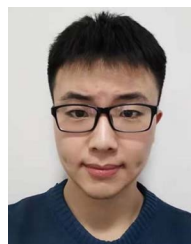
If the conditions in Lemma 2 are all satisfied and ℓ_u is neglected, $\mathbf{r}_3$ will be zero. It is obvious that the uncertainty estimated error $\tilde{\Delta}$ can converge to zero as $t \rightarrow \infty$.

Moreover, if ℓ_u is considered, $\tilde{\Delta}$ will converge to a bounded set $\sqrt{\frac{\|C\mathbf{r}_3\|^2}{|\delta|}}$, which can be arbitrarily small by increasing absolute value of the eigenvalues of observer. ■

REFERENCES

- [1] G. C. Goodwin and K. S. Sin, *Adaptive Filtering Prediction and Control*. Newburyport, MA, USA: Dover Publications, 2014.
- [2] I. S. Khalil, J. Doyle, and K. Glover, *Robust and Optimal Control*. Hoboken, NJ, USA: Prentice Hall, 1996.
- [3] J.-M. Yang and J.-H. Kim, "Sliding mode control for trajectory tracking of nonholonomic wheeled mobile robots," *IEEE Trans. Robot. Autom.*, vol. 15, no. 3, pp. 578–587, Jun. 1999.
- [4] R. Sanner and J.-J. Slotine, "Gaussian networks for direct adaptive control," *IEEE Trans. Neural Netw.*, vol. 3, no. 6, pp. 837–863, Nov. 1992.
- [5] L. Guo and W. Chen, "Disturbance attenuation and rejection for systems with nonlinearity via DOBC approach," *Int. J. Robust Nonlinear Control*, vol. 15, no. 3, pp. 109–125, 2005.
- [6] J. Jia, K. Guo, X. Yu, L. Guo, and L. Xie, "Agile flight control under multiple disturbances for quadrotor: Algorithms and evaluation," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 58, no. 4, pp. 3049–3062, Aug. 2022.
- [7] S. C. Hoi, D. Sahoo, J. Lu, and P. Zhao, "Online learning: A comprehensive survey," *Neurocomputing*, vol. 459, pp. 249–289, 2021.
- [8] P. Mitran, D. McConachie, and D. Berenson, "Learning where to trust unreliable models in an unstructured world for deformable object manipulation," *Sci. Robot.*, vol. 6, no. 54, 2021, Art. no. eabd8170.
- [9] H.-T. Nguyen, C. C. Cheah, and K.-A. Toh, "An analytic layer-wise deep learning framework with applications to robotics," *Automatica*, vol. 135, 2022, Art. no. 110007.
- [10] D. A. Markov, L. Petruccio, A. M. Kist, and R. Portugues, "A cerebellar internal model calibrates a feedback controller involved in sensorimotor control," *Nature Commun.*, vol. 12, no. 1, pp. 1–21, 2021.
- [11] E. Sariyildiz, R. Oboe, and K. Ohnishi, "Disturbance observer-based robust control and its applications: 35th anniversary overview," *IEEE Trans. Ind. Electron.*, vol. 67, no. 3, pp. 2042–2053, Mar. 2020.
- [12] C. P. Bechlioulis and G. A. Rovithakis, "Robust partial-state feedback prescribed performance control of cascade systems with unknown nonlinearities," *IEEE Trans. Autom. Control*, vol. 56, no. 9, pp. 2224–2230, Sep. 2011.
- [13] W. Chen, J. Yang, L. Guo, and S. Li, "Disturbance-observer-based control and related methods—An overview," *IEEE Trans. Ind. Electron.*, vol. 63, no. 2, pp. 1083–1095, Feb. 2016.
- [14] X. Li, Q.-G. Wang, X. Li, K. K. Tan, and L. Xie, "Feedforward control with disturbance prediction for linear discrete-time systems," *IEEE Trans. Control Syst. Technol.*, vol. 27, no. 6, pp. 2340–2350, Nov. 2019.
- [15] K. Ohishi, M. Nakao, K. Ohnishi, and K. Miyachi, "Microprocessor-controlled DC motor for load-insensitive position servo system," *IEEE Trans. Ind. Electron.*, vol. 34, no. 1, pp. 44–49, Feb. 1987.
- [16] J. Han, "From PID to active disturbance rejection control," *IEEE Trans. Ind. Electron.*, vol. 56, no. 3, pp. 900–906, Mar. 2009.
- [17] C. Johnson, "Further study of the linear regulator with disturbances—the case of vector disturbances satisfying a linear differential equation," *IEEE Trans. Autom. Control*, vol. 15, no. 2, pp. 222–228, Apr. 1970.
- [18] H. S.-Ramirez and M. A. O.-Salazar, "On the robust control of buck-converter DC-motor combinations," *IEEE Trans. Power Electron.*, vol. 28, no. 8, pp. 3912–3922, Aug. 2013.
- [19] L. B. Freidovich and H. K. Khalil, "Performance recovery of feedback-linearization-based designs," *IEEE Trans. Autom. Control*, vol. 53, no. 10, pp. 2324–2334, Nov. 2008.
- [20] W. Chen, D. Ballance, P. Gawthrop, and J. O'Reilly, "A nonlinear disturbance observer for robotic manipulators," *IEEE Trans. Ind. Electron.*, vol. 47, no. 4, pp. 932–938, Aug. 2000.
- [21] W. Chen, "Disturbance observer based control for nonlinear systems," *IEEE/ASME Trans. Mechatron.*, vol. 9, no. 4, pp. 706–710, Dec. 2004.
- [22] K.-S. Kim, K.-H. Rew, and S. Kim, "Disturbance observer for estimating higher order disturbances in time series expansion," *IEEE Trans. Autom. Control*, vol. 55, no. 8, pp. 1905–1911, Aug. 2010.

- [23] M. Bisheban and T. Lee, "Geometric adaptive control with neural networks for a quadrotor in wind fields," *IEEE Trans. Control Syst. Technol.*, vol. 29, no. 4, pp. 1533–1548, Jul. 2021.
- [24] W. Chen and L. Guo, "Analysis of disturbance observer based control for nonlinear systems under disturbances with bounded variation," in *Proc. Int. Conf. Control*, 2004, pp. 1–5.
- [25] Z. Gao, "Scaling and bandwidth-parameterization based controller tuning," in *Proc. Amer. Control Conf.*, 2006, pp. 4989–4996.
- [26] S. Singh, S. M. Richards, V. Sindhvani, J.-J. E. Slotine, and M. Pavone, "Learning stabilizable nonlinear dynamics with contraction-based regularization," *Int. J. Robot. Res.*, vol. 40, no. 10–11, pp. 1123–1150, 2021.
- [27] M. P. Deisenroth, D. Fox, and C. E. Rasmussen, "Gaussian processes for data-efficient learning in robotics and control," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 2, pp. 408–423, Feb. 2015.
- [28] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser, "TossingBot: Learning to throw arbitrary objects with residual physics," *IEEE Trans. Robot.*, vol. 36, no. 4, pp. 1307–1319, Aug. 2020.
- [29] K. Karydis, I. Poulakakis, J. Sun, and H. G. Tanner, "Probabilistically valid stochastic extensions of deterministic models for systems with uncertainty," *Int. J. Robot. Res.*, vol. 34, no. 10, pp. 1278–1295, 2015.
- [30] V. R. Desaraju, A. E. Spitzer, C. O'Meadhra, L. Lieu, and N. Michael, "Leveraging experience for robust, adaptive nonlinear MPC on computationally constrained systems with time-varying state uncertainty," *Int. J. Robot. Res.*, vol. 37, no. 13–14, pp. 1690–1712, 2018.
- [31] T. Johannink et al., "Residual reinforcement learning for robot control," in *Proc. Int. Conf. Robot. Automat.*, 2019, pp. 6023–6029.
- [32] M. Saveriano, Y. Yin, P. Falco, and D. Lee, "Data-efficient control policy search using residual dynamics learning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2017, pp. 4709–4715.
- [33] M. Zheng, X. Lyu, X. Liang, and F. Zhang, "A generalized design method for learning-based disturbance observer," *IEEE/ASME Trans. Mechatron.*, vol. 26, no. 1, pp. 45–54, Feb. 2021.
- [34] J.-M. Kai, G. Allibert, M.-D. Hua, and T. Hamel, "Nonlinear feedback control of quadrotors exploiting first-order drag effects," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 8189–8195, 2017.
- [35] M. Faessler, A. Franchi, and D. Scaramuzza, "Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories," *IEEE Robot. Autom. Lett.*, vol. 3, no. 2, pp. 620–626, Apr. 2018.
- [36] J. Jia, K. Guo, X. Yu, W. Zhao, and L. Guo, "Accurate high-maneuvering trajectory tracking for quadrotors: A drag utilization method," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 6966–6973, Jul. 2022.
- [37] B. O. Koopman, "Hamiltonian systems and transformation in Hilbert space," *Proc. Nat. Acad. Sci.*, vol. 17, no. 5, 1931, Art. no. 315.
- [38] A. Mesbahi, J. Bu, and M. Mesbahi, "Nonlinear observability via Koopman analysis: Characterizing the role of symmetry," *Automatica*, vol. 124, 2021, Art. no. 109353.
- [39] A. Surana, "Koopman operator based observer synthesis for control-affine nonlinear systems," in *Proc. IEEE 55th Conf. Decis. Control*, 2016, pp. 6492–6499.
- [40] A. Surana and A. Banaszuk, "Linear observer synthesis for nonlinear systems using Koopman operator framework," *IFAC-PapersOnLine*, vol. 49, no. 18, pp. 716–723, 2016.
- [41] A. Broad, I. Abraham, T. Murphey, and B. Argall, "Data-driven Koopman operators for model-based shared control of human-machine systems," *Int. J. Robot. Res.*, vol. 39, no. 9, pp. 1178–1195, 2020.
- [42] J. L. Proctor, S. L. Brunton, and J. N. Kutz, "Generalizing Koopman theory to allow for inputs and control," *SIAM J. Appl. Dynamical Syst.*, vol. 17, no. 1, pp. 909–930, 2018.
- [43] I. Mezić, "On applications of the spectral theory of the Koopman operator in dynamical systems and control theory," in *Proc. IEEE 54th Conf. Decis. Control*, 2015, pp. 7034–7041.
- [44] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley, "A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition," *J. Nonlinear Sci.*, vol. 25, no. 6, pp. 1307–1346, 2015.
- [45] J. N. Kutz, S. L. Brunton, B. W. Brunton, and J. L. Proctor, *Dynamic Mode Decomposition: Data-Driven Modeling of Complex Systems*. Philadelphia, PA, USA: SIAM, 2016.
- [46] I. Abraham and T. D. Murphey, "Active learning of dynamics for data-driven control using Koopman operators," *IEEE Trans. Robot.*, vol. 35, no. 5, pp. 1071–1083, Oct. 2019.
- [47] G. Mamakoukas, M. L. Castaño, X. Tan, and T. D. Murphey, "Derivative-based Koopman operators for real-time control of robotic systems," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 2173–2192, Dec. 2021.
- [48] D. Bruder, X. Fu, R. B. Gillespie, C. D. Remy, and R. Vasudevan, "Data-driven control of soft robots using Koopman operator theory," *IEEE Trans. Robot.*, vol. 37, no. 3, pp. 948–961, Jun. 2021.
- [49] M. Haseli and J. Cortés, "Learning Koopman eigenfunctions and invariant subspaces from data: Symmetric subspace decomposition," *IEEE Trans. Autom. Control*, vol. 67, no. 7, pp. 3442–3457, Jul. 2022.
- [50] M. Korda and I. Mezić, "On convergence of extended dynamic mode decomposition to the Koopman operator," *J. Nonlinear Sci.*, vol. 28, no. 2, pp. 687–710, 2018.
- [51] S. Klus, P. Koltai, and C. Schütte, "On the numerical approximation of the Perron-Frobenius and Koopman operator," *J. Comput. Dyn.*, vol. 3, no. 1, pp. 51–79, 2016.
- [52] L. Shi and K. Karydis, "ACD-EDMD: Analytical construction for dictionaries of lifting functions in Koopman operator-based nonlinear robotic systems," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 906–913, Apr. 2022.
- [53] H. Shi and M. Q.-H. Meng, "Deep Koopman operator with control for nonlinear systems," *IEEE Robot. Autom. Lett.*, vol. 7, no. 3, pp. 7700–7707, Jul. 2022.
- [54] J. Woolfrey, W. Lu, and D. Liu, "Predictive end-effector control of manipulators on moving platforms under disturbance," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 2210–2217, Dec. 2021.
- [55] M. Gupta, J. Gao, C. C. Aggarwal, and J. Han, "Outlier detection for temporal data: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 26, no. 9, pp. 2250–2267, Sep. 2014.
- [56] R. Seeber, H. Haimovich, M. Horn, L. M. Fridman, and H. De Battista, "Robust exact differentiators with predefined convergence time," *Automatica*, vol. 134, 2021, Art. no. 109858.
- [57] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proc. Nat. Acad. Sci.*, vol. 113, no. 15, pp. 3932–3937, 2016.
- [58] E. Celeghini, M. Gadella, and M. A. del Olmo, "Hermite functions and Fourier series," *Symmetry*, vol. 13, no. 5, 2021, Art. no. 853.
- [59] U. Frese et al., "Off-the-shelf vision for a robotic ball catcher," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2001, pp. 1623–1629.
- [60] M. Müller, S. Lupashin, and R. D'Andrea, "Quadrocopter ball juggling," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2011, pp. 5113–5120.
- [61] P. Bouffard, A. Aswani, and C. Tomlin, "Learning-based model predictive control on a quadrotor: Onboard implementation and experimental results," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2012, pp. 279–284.
- [62] S. Kim and A. Billard, "Estimating the non-linear dynamics of free-flying objects," *Robot. Auton. Syst.*, vol. 60, no. 9, pp. 1108–1122, 2012.
- [63] S. Kim, A. Shukla, and A. Billard, "Catching objects in flight," *IEEE Trans. Robot.*, vol. 30, no. 5, pp. 1049–1065, Oct. 2014.
- [64] C.-C. Chang and C.-J. Lin, "LIBSVM: A library for support vector machines," *ACM Trans. Intell. Syst. Technol.*, vol. 2, no. 3, pp. 1–27, 2011.
- [65] Y. Jia, M. Gardner, and X. Mu, "Batting an in-flight object to the target," *Int. J. Robot. Res.*, vol. 38, no. 4, pp. 451–485, 2019.
- [66] M. W. Mueller, M. Hehn, and R. D'Andrea, "A computationally efficient motion primitive for quadrocopter trajectory generation," *IEEE Trans. Robot.*, vol. 31, no. 6, pp. 1294–1310, Dec. 2015.
- [67] H. Yu et al., "Neural motion prediction for in-flight uneven object catching," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2021, pp. 4662–4669.
- [68] M. Watterson and V. Kumar, "Control of quadrotors using the Hopf fibration on $SO(3)$," in *Proc. Robot. Res.*, 2020, pp. 199–215.
- [69] M. O'Connell et al., "Neural-fly enables rapid learning for agile flight in strong winds," *Sci. Robot.*, vol. 7, no. 66, 2022, Art. no. eabm6597.



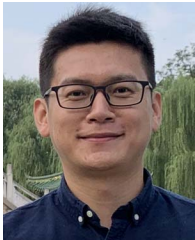
Jindou Jia received the B.S. degree in automation from Harbin Engineering University, Harbin, China, in 2019. He is currently working toward the Ph.D. degree in control theory and control engineering with the School of Automation Science and Electrical Engineering, Beihang University, Beijing, China.

His current research interests include accurate control and motion planning for robots.



Wenyu Zhang was born in Bozhou, China, in 1999. He received the B.S. degree in aircraft control and information engineering from Northwestern Polytechnical University, Xi'an, China, in 2020, and the M.S. degree in control science and engineering from the School of Automation Science and Electrical Engineering, Beihang University, Beijing, China.

He is currently working with ZTE.



Kexin Guo received the B.S. degree in control theory and control engineering from Beijing Institute of Technology, Beijing, China, in 2011, the M.S. degree in instrumentation science and technology from Beihang University, Beijing, in 2014, and the Ph.D. degree in electrical and electronic engineering from Nanyang Technological University, Singapore, in 2018.

He was a Postdoctoral Fellow with the "Zhuoyue" program with the School of Automation Science and Electrical Engineering, Beihang University, in 2019.

Since 2021, he has been an Associate Professor with the School of Aeronautics Science and Engineering, Beihang University, Beijing. His current research interests include GNSS-denied localization, bio-inspired control, and safety control of unmanned aerial vehicles.

Dr. Guo was the recipient of the Best Paper and Best Paper Finalist at international conferences.



Jianliang Wang (Senior Member, IEEE) received the B.E. degree from the Beijing Institute of Technology, Beijing, China, in 1982, and the M.S.E., and Ph.D. degrees from the Johns Hopkins University, Baltimore, MD, USA, in 1985 and 1988, respectively, all in electrical engineering.

From 1988 to 1990, he was a Lecturer with Beihang University. From 1990 to 2019, he was a tenured Associate Professor with Nanyang Technological University, Singapore. Since 2020, he has been a Senior Researcher/Professor with the Hangzhou Innovation

Institute of Beihang University, Beijing, China. His current research interests include multi-agent systems, UAVs and UAMs, vehicular platoon systems, and fault tolerant control.

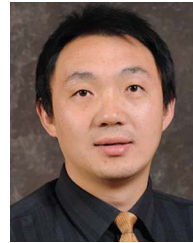


Xiang Yu (Senior Member, IEEE) received the B.S., M.S., and Ph.D. degrees in automation science and engineering from Northwestern Polytechnical University, Xi'an, China, in 2003, 2004, and 2008, respectively.

He is currently a Professor with the School of Automation Science and Electrical Engineering, Beihang University, Beijing, China. He was a Postdoctoral Research Fellow with the Department of Electrical and Computer Engineering, Western University, London, ON, Canada, and a Research Associate with

the Department of Mechanical, Industrial and Aerospace Engineering, Concordia University, Montreal, QC, Canada. He has authored over 70 prestigious journal papers and a monograph, including IEEE transactions, *AIAA Journal*, and *Progress in Aerospace Sciences*. His current research interests include safety control of unmanned aerial vehicles, autonomous navigation, and control of hypersonic vehicles.

Prof. Yu was a recipient of the Recruitment Program for Young Professionals, First Prize of Science and Technology Progress Award of China Instrument and Control Society, Youth Science and Technology Award of Chinese Society of Aeronautics and Astronautics, and the Best Paper and Best Paper Finalist at international conferences. He has also been the Associate Editor of *Journal of Field Robotics*, *Journal of Intelligent & Robotic Systems*, *Asian Journal of Control*, and *Chinese Journal of Aeronautics*. He has been the Program Co-Chair, Invitation Chair, and IPC Member of several academic conferences.



Yang Shi (Fellow, IEEE) received the B.Sc. and Ph.D. degrees in mechanical engineering and automatic control from Northwestern Polytechnical University, Xi'an, China, in 1994 and 1998, respectively, and the Ph.D. degree in electrical and computer engineering from the University of Alberta, Edmonton, AB, Canada, in 2005.

From 2005 to 2009, he was an Assistant Professor and an Associate Professor with the Department of Mechanical Engineering, University of Saskatchewan, Saskatoon, SK, Canada. In 2009, he

joined the University of Victoria, and currently he is a Professor with the Department of Mechanical Engineering, University of Victoria, Victoria, BC, Canada. His current research interests include networked and distributed systems, model predictive control (MPC), cyber-physical systems (CPS), robotics and mechatronics, navigation and control of autonomous systems (AUV and UAV), and energy system applications.

Dr. Shi was the recipient of the University of Saskatchewan Student Union Teaching Excellence Award in 2007, and the Faculty of Engineering Teaching Excellence Award in 2012, the University of Victoria (UVic), the UVic Craigdarroch Silver Medal for Excellence in Research in 2015, the 2017 IEEE Transactions on Fuzzy Systems Outstanding Paper Award, the Humboldt Research Fellowship for Experienced Researchers in 2018; CSME Mechatronics Medal (2023); IEEE Dr.-Ing. Eugene Mittelmann Achievement Award (2023). He was also the recipient of the JSPS Invitation Fellowship (short-term) in 2013. He is IFAC Council Member, VP on conference activities with IEEE IES, and the Chair of IEEE IES Technical Committee on Industrial Cyber-Physical Systems. Currently, he is a co-Editor-in-Chief of IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, and Editor-in-Chief of IEEE CANADIAN JOURNAL OF ELECTRICAL AND COMPUTER ENGINEERING. He also serves as Associate Editor for *Automatica*, IEEE TRANSACTIONS ON AUTOMATIC CONTROL, *Annual Review in Controls*, etc. He is a Distinguished Lecturer of IES. He is a Fellow of ASME, CSME, Engineering Institute of Canada (EIC), Canadian Academy of Engineering (CAE), and a registered Professional Engineer in British Columbia, Canada.



Lei Guo (Fellow, IEEE) was born in Qufu, China, in 1966. He received the B.S. degree in fundamental mathematics and M.S. degree in operational research and cybernetics from Qufu Normal University, Qufu, China, in 1988 and 1991, respectively, and the Ph.D. degree in control engineering from Southeast University, Nanjing, China, in 1997.

From 1991 to 1994, he was a Lecturer with Qingdao University, Qingdao, China. From 1997 to 1999, he was a Postdoctoral Fellow with Southeast University.

From 1999 to 2000, he was a Research Fellow with IRCCyN, Nantes, France. From 2000 to 2003, he was a Research Fellow with the University of Glasgow, Glasgow, U.K., Loughborough University, Loughborough, U.K., and the University of Manchester Institute of Science and Technology, Manchester, U.K. In 2004, he was a Professor with the Institute of Automation, Southeast University. In 2006, he was a Professor with the School of Instrumentation and Opto-Electronics Engineering, Beihang University, Beijing, China, where he is currently with the School of Automation Science and Electronic Engineering. He has authored or coauthored more than 260 papers and served as an Editor for five journals. His research interests include robust control, stochastic systems, fault detection, filter design, and nonlinear control with their applications to aerospace systems.

Dr. Guo is a recipient of an Award of the National Science Fund for Distinguished Young Scholars of China and is a Changjiang Distinguished Professor, Ministry of Education of China.